
Virtual Memory

- 9.1 Physical and Virtual Addressing 803
- 9.2 Address Spaces 804
- 9.3 VM as a Tool for Caching 805
- 9.4 VM as a Tool for Memory Management 811
- 9.5 VM as a Tool for Memory Protection 812
- 9.6 Address Translation 813
- 9.7 Case Study: The Intel Core i7/Linux Memory System 825
- 9.8 Memory Mapping 833
- 9.9 Dynamic Memory Allocation 839
- 9.10 Garbage Collection 865
- 9.11 Common Memory-Related Bugs in C Programs 870
- 9.12 Summary 875
- Bibliographic Notes 876
- Homework Problems 876
- Solutions to Practice Problems 880

Processes in a system share the CPU and main memory with other processes. However, sharing the main memory poses some special challenges. As demand on the CPU increases, processes slow down in some reasonably smooth way. But if too many processes need too much memory, then some of them will simply not be able to run. When a program is out of space, it is out of luck. Memory is also vulnerable to corruption. If some process inadvertently writes to the memory used by another process, that process might fail in some bewildering fashion totally unrelated to the program logic.

In order to manage memory more efficiently and with fewer errors, modern systems provide an abstraction of main memory known as *virtual memory (VM)*. Virtual memory is an elegant interaction of hardware exceptions, hardware address translation, main memory, disk files, and kernel software that provides each process with a large, uniform, and private address space. With one clean mechanism, virtual memory provides three important capabilities: (1) It uses main memory efficiently by treating it as a cache for an address space stored on disk, keeping only the active areas in main memory and transferring data back and forth between disk and memory as needed. (2) It simplifies memory management by providing each process with a uniform address space. (3) It protects the address space of each process from corruption by other processes.

Virtual memory is one of the great ideas in computer systems. A major reason for its success is that it works silently and automatically, without any intervention from the application programmer. Since virtual memory works so well behind the scenes, why would a programmer need to understand it? There are several reasons.

- *Virtual memory is central.* Virtual memory pervades all levels of computer systems, playing key roles in the design of hardware exceptions, assemblers, linkers, loaders, shared objects, files, and processes. Understanding virtual memory will help you better understand how systems work in general.
- *Virtual memory is powerful.* Virtual memory gives applications powerful capabilities to create and destroy chunks of memory, map chunks of memory to portions of disk files, and share memory with other processes. For example, did you know that you can read or modify the contents of a disk file by reading and writing memory locations? Or that you can load the contents of a file into memory without doing any explicit copying? Understanding virtual memory will help you harness its powerful capabilities in your applications.
- *Virtual memory is dangerous.* Applications interact with virtual memory every time they reference a variable, dereference a pointer, or make a call to a dynamic allocation package such as `malloc`. If virtual memory is used improperly, applications can suffer from perplexing and insidious memory-related bugs. For example, a program with a bad pointer can crash immediately with a “segmentation fault” or a “protection fault,” run silently for hours before crashing, or scariest of all, run to completion with incorrect results. Understanding virtual memory, and the allocation packages such as `malloc` that manage it, can help you avoid these errors.

This chapter looks at virtual memory from two angles. The first half of the chapter describes how virtual memory works. The second half describes how virtual memory is used and managed by applications. There is no avoiding the fact that VM is complicated, and the discussion reflects this in places. The good news is that if you work through the details, you will be able to simulate the virtual memory mechanism of a small system by hand, and the virtual memory idea will be forever demystified.

The second half builds on this understanding, showing you how to use and manage virtual memory in your programs. You will learn how to manage virtual memory via explicit memory mapping and calls to dynamic storage allocators such as the `malloc` package. You will also learn about a host of common memory-related errors in C programs and how to avoid them.

9.1 Physical and Virtual Addressing

The main memory of a computer system is organized as an array of M contiguous byte-size cells. Each byte has a unique *physical address* (*PA*). The first byte has an address of 0, the next byte an address of 1, the next byte an address of 2, and so on. Given this simple organization, the most natural way for a CPU to access memory would be to use physical addresses. We call this approach *physical addressing*. Figure 9.1 shows an example of physical addressing in the context of a load instruction that reads the 4-byte word starting at physical address 4. When the CPU executes the load instruction, it generates an effective physical address and passes it to main memory over the memory bus. The main memory fetches the 4-byte word starting at physical address 4 and returns it to the CPU, which stores it in a register.

Early PCs used physical addressing, and systems such as digital signal processors, embedded microcontrollers, and Cray supercomputers continue to do so. However, modern processors use a form of addressing known as *virtual addressing*, as shown in Figure 9.2.

Figure 9.1
A system that uses
physical addressing.

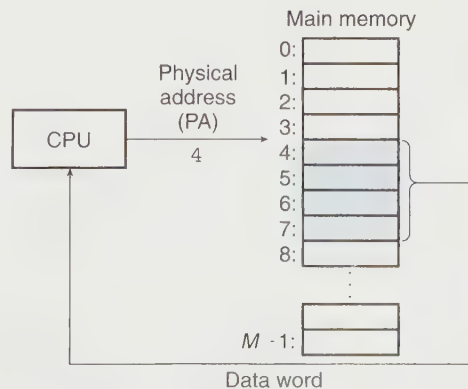
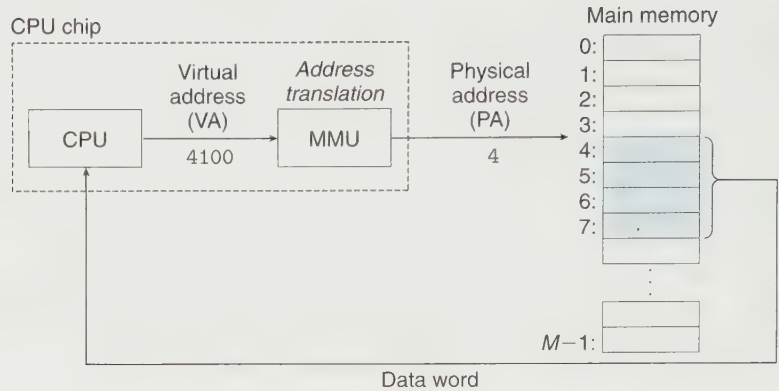


Figure 9.2
A system that uses virtual addressing.



With virtual addressing, the CPU accesses main memory by generating a *virtual address (VA)*, which is converted to the appropriate physical address before being sent to main memory. The task of converting a virtual address to a physical one is known as *address translation*. Like exception handling, address translation requires close cooperation between the CPU hardware and the operating system. Dedicated hardware on the CPU chip called the *memory management unit (MMU)* translates virtual addresses on the fly, using a lookup table stored in main memory whose contents are managed by the operating system.

9.2 Address Spaces

An *address space* is an ordered set of nonnegative integer addresses

$$\{0, 1, 2, \dots\}$$

If the integers in the address space are consecutive, then we say that it is a *linear address space*. To simplify our discussion, we will always assume linear address spaces. In a system with virtual memory, the CPU generates virtual addresses from an address space of $N = 2^n$ addresses called the *virtual address space*:

$$\{0, 1, 2, \dots, N - 1\}$$

The size of an address space is characterized by the number of bits that are needed to represent the largest address. For example, a virtual address space with $N = 2^n$ addresses is called an n -bit address space. Modern systems typically support either 32-bit or 64-bit virtual address spaces.

A system also has a *physical address space* that corresponds to the M bytes of physical memory in the system:

$$\{0, 1, 2, \dots, M - 1\}$$

M is not required to be a power of 2, but to simplify the discussion, we will assume that $M = 2^m$.

The concept of an address space is important because it makes a clean distinction between data objects (bytes) and their attributes (addresses). Once we recognize this distinction, then we can generalize and allow each data object to have multiple independent addresses, each chosen from a different address space. This is the basic idea of virtual memory. Each byte of main memory has a virtual address chosen from the virtual address space, and a physical address chosen from the physical address space.

Practice Problem 9.1 (solution page 880)

Complete the following table, filling in the missing entries and replacing each question mark with the appropriate integer. Use the following units: K = 2¹⁰ (kilo), M = 2²⁰ (mega), G = 2³⁰ (giga), T = 2⁴⁰ (tera), P = 2⁵⁰ (peta), or E = 2⁶⁰ (exa).

Number of virtual address bits (<i>n</i>)	Number of virtual addresses (<i>N</i>)	Largest possible virtual address
8	2 [?] = 64 K	
		2 ³² − 1 = ? G − 1
	2 [?] = 256 T	
64		

9.3 VM as a Tool for Caching

Conceptually, a virtual memory is organized as an array of *N* contiguous byte-size cells stored on disk. Each byte has a unique virtual address that serves as an index into the array. The contents of the array on disk are cached in main memory. As with any other cache in the memory hierarchy, the data on disk (the lower level) is partitioned into blocks that serve as the transfer units between the disk and the main memory (the upper level). VM systems handle this by partitioning the virtual memory into fixed-size blocks called *virtual pages* (*VPs*). Each virtual page is *P* = 2^{*p*} bytes in size. Similarly, physical memory is partitioned into *physical pages* (*PPs*), also *P* bytes in size. (Physical pages are also referred to as *page frames*.)

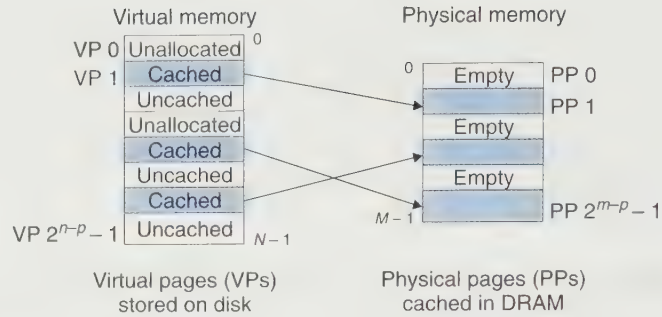
At any point in time, the set of virtual pages is partitioned into three disjoint subsets:

- Unallocated.* Pages that have not yet been allocated (or created) by the VM system. Unallocated blocks do not have any data associated with them, and thus do not occupy any space on disk.
- Cached.* Allocated pages that are currently cached in physical memory.
- Uncached.* Allocated pages that are not cached in physical memory.

The example in Figure 9.3 shows a small virtual memory with eight virtual pages. Virtual pages 0 and 3 have not been allocated yet, and thus do not yet exist

Figure 9.3

How a VM system uses main memory as a cache.



on disk. Virtual pages 1, 4, and 6 are cached in physical memory. Pages 2, 5, and 7 are allocated but are not currently cached in physical memory.

9.3.1 DRAM Cache Organization

To help us keep the different caches in the memory hierarchy straight, we will use the term *SRAM cache* to denote the L1, L2, and L3 cache memories between the CPU and main memory, and the term *DRAM cache* to denote the VM system's cache that caches virtual pages in main memory.

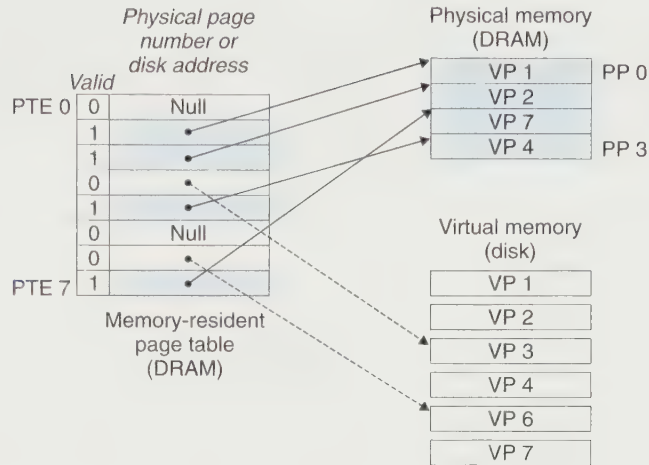
The position of the DRAM cache in the memory hierarchy has a big impact on the way that it is organized. Recall that a DRAM is at least 10 times slower than an SRAM and that disk is about 100,000 times slower than a DRAM. Thus, misses in DRAM caches are very expensive compared to misses in SRAM caches because DRAM cache misses are served from disk, while SRAM cache misses are usually served from DRAM-based main memory. Further, the cost of reading the first byte from a disk sector is about 100,000 times slower than reading successive bytes in the sector. The bottom line is that the organization of the DRAM cache is driven entirely by the enormous cost of misses.

Because of the large miss penalty and the expense of accessing the first byte, virtual pages tend to be large—typically 4 KB to 2 MB. Due to the large miss penalty, DRAM caches are fully associative; that is, any virtual page can be placed in any physical page. The replacement policy on misses also assumes greater importance, because the penalty associated with replacing the wrong virtual page is so high. Thus, operating systems use much more sophisticated replacement algorithms for DRAM caches than the hardware does for SRAM caches. (These replacement algorithms are beyond our scope here.) Finally, because of the large access time of disk, DRAM caches always use write-back instead of write-through.

9.3.2 Page Tables

As with any cache, the VM system must have some way to determine if a virtual page is cached somewhere in DRAM. If so, the system must determine which physical page it is cached in. If there is a miss, the system must determine

Figure 9.4
Page table.



where the virtual page is stored on disk, select a victim page in physical memory, and copy the virtual page from disk to DRAM, replacing the victim page.

These capabilities are provided by a combination of operating system software, address translation hardware in the MMU (memory management unit), and a data structure stored in physical memory known as a *page table* that maps virtual pages to physical pages. The address translation hardware reads the page table each time it converts a virtual address to a physical address. The operating system is responsible for maintaining the contents of the page table and transferring pages back and forth between disk and DRAM.

Figure 9.4 shows the basic organization of a page table. A page table is an array of *page table entries* (PTEs). Each page in the virtual address space has a PTE at a fixed offset in the page table. For our purposes, we will assume that each PTE consists of a *valid bit* and an *n*-bit address field. The valid bit indicates whether the virtual page is currently cached in DRAM. If the valid bit is set, the address field indicates the start of the corresponding physical page in DRAM where the virtual page is cached. If the valid bit is not set, then a null address indicates that the virtual page has not yet been allocated. Otherwise, the address points to the start of the virtual page on disk.

The example in Figure 9.4 shows a page table for a system with eight virtual pages and four physical pages. Four virtual pages (VP 1, VP 2, VP 4, and VP 7) are currently cached in DRAM. Two pages (VP 0 and VP 5) have not yet been allocated, and the rest (VP 3 and VP 6) have been allocated but are not currently cached. An important point to notice about Figure 9.4 is that because the DRAM cache is fully associative, any physical page can contain any virtual page.

Practice Problem 9.2 (solution page 881)

Determine the number of page table entries (PTEs) that are needed for the following combinations of virtual address size (*n*) and page size (*P*):

<i>n</i>	$P = 2^p$	Number of PTEs
16	4K	
16	8K	
32	4K	
32	8K	

9.3.3 Page Hits

Consider what happens when the CPU reads a word of virtual memory contained in VP 2, which is cached in DRAM (Figure 9.5). Using a technique we will describe in detail in Section 9.6, the address translation hardware uses the virtual address as an index to locate PTE 2 and read it from memory. Since the valid bit is set, the address translation hardware knows that VP 2 is cached in memory. So it uses the physical memory address in the PTE (which points to the start of the cached page in PP 1) to construct the physical address of the word.

9.3.4 Page Faults

In virtual memory parlance, a DRAM cache miss is known as a *page fault*. Figure 9.6 shows the state of our example page table before the fault. The CPU has referenced a word in VP 3, which is not cached in DRAM. The address translation hardware reads PTE 3 from memory, infers from the valid bit that VP 3 is not cached, and triggers a page fault exception. The page fault exception invokes a page fault exception handler in the kernel, which selects a victim page—in this case, VP 4 stored in PP 3. If VP 4 has been modified, then the kernel copies it back to disk. In either case, the kernel modifies the page table entry for VP 4 to reflect the fact that VP 4 is no longer cached in main memory.

Figure 9.5
VM page hit. The reference to a word in VP 2 is a hit.

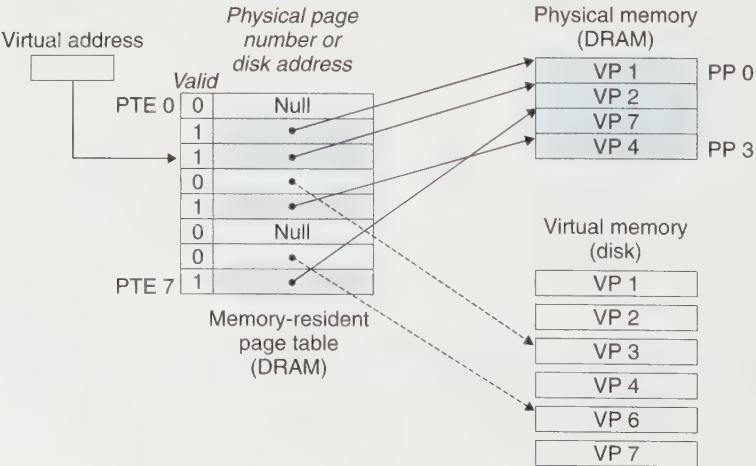
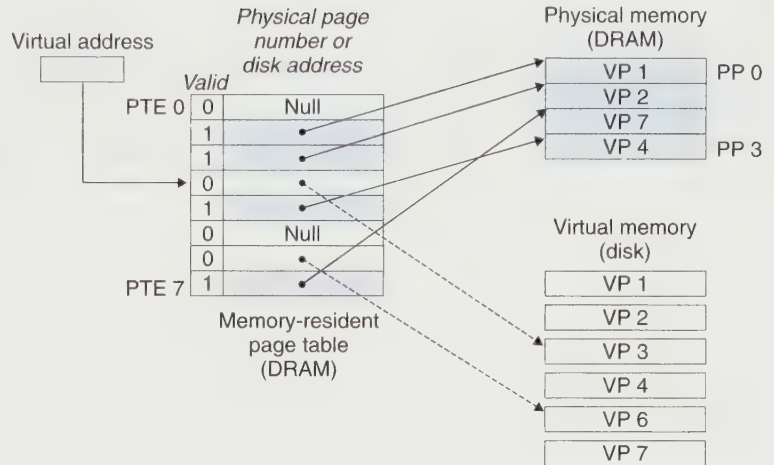
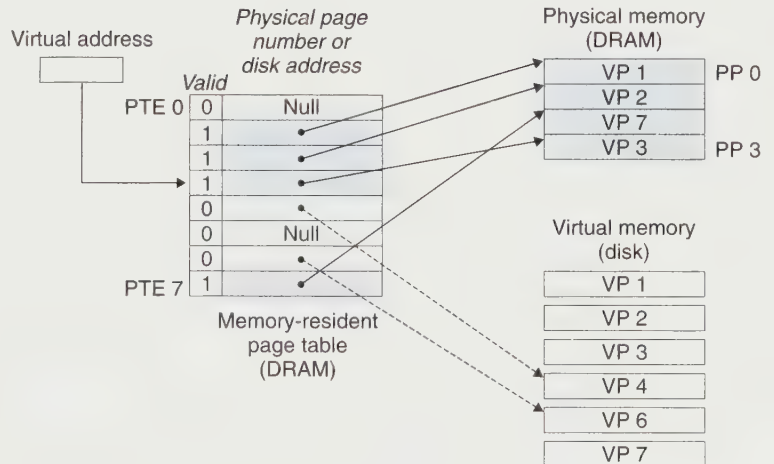


Figure 9.6

VM page fault (before).
The reference to a word in VP 3 is a miss and triggers a page fault.

**Figure 9.7**

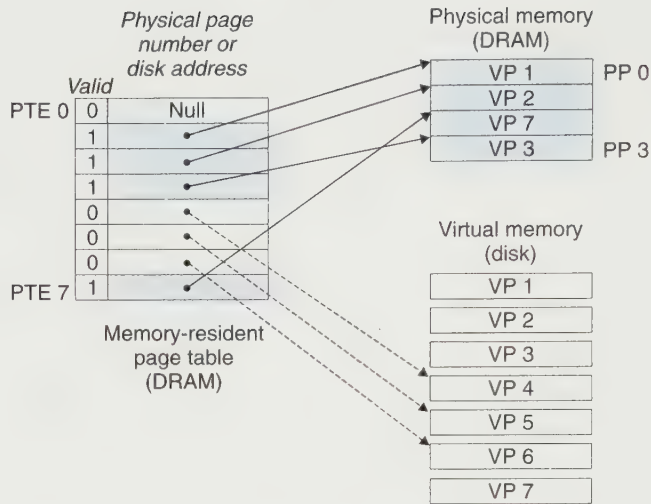
VM page fault (after).
The page fault handler selects VP 4 as the victim and replaces it with a copy of VP 3 from disk. After the page fault handler restarts the faulting instruction, it will read the word from memory normally, without generating an exception.



Next, the kernel copies VP 3 from disk to PP 3 in memory, updates PTE 3, and then returns. When the handler returns, it restarts the faulting instruction, which resends the faulting virtual address to the address translation hardware. But now, VP 3 is cached in main memory, and the page hit is handled normally by the address translation hardware. Figure 9.7 shows the state of our example page table after the page fault.

Virtual memory was invented in the early 1960s, long before the widening CPU-memory gap spawned SRAM caches. As a result, virtual memory systems use a different terminology from SRAM caches, even though many of the ideas are similar. In virtual memory parlance, blocks are known as pages. The activity of transferring a page between disk and memory is known as *swapping* or *paging*. Pages are *swapped in* (*paged in*) from disk to DRAM, and *swapped out* (*paged out*) from DRAM to disk. The strategy of waiting until the last moment to swap

Figure 9.8
Allocating a new virtual page. The kernel allocates VP 5 on disk and points PTE 5 to this new location.



in a page, when a miss occurs, is known as *demand paging*. Other approaches, such as trying to predict misses and swap pages in before they are actually referenced, are possible. However, all modern systems use demand paging.

9.3.5 Allocating Pages

Figure 9.8 shows the effect on our example page table when the operating system allocates a new page of virtual memory—for example, as a result of calling `malloc`. In the example, VP 5 is allocated by creating room on disk and updating PTE 5 to point to the newly created page on disk.

9.3.6 Locality to the Rescue Again

When many of us learn about the idea of virtual memory, our first impression is often that it must be terribly inefficient. Given the large miss penalties, we worry that paging will destroy program performance. In practice, virtual memory works well, mainly because of our old friend *locality*.

Although the total number of distinct pages that programs reference during an entire run might exceed the total size of physical memory, the principle of locality promises that at any point in time they will tend to work on a smaller set of *active pages* known as the *working set* or *resident set*. After an initial overhead where the working set is paged into memory, subsequent references to the working set result in hits, with no additional disk traffic.

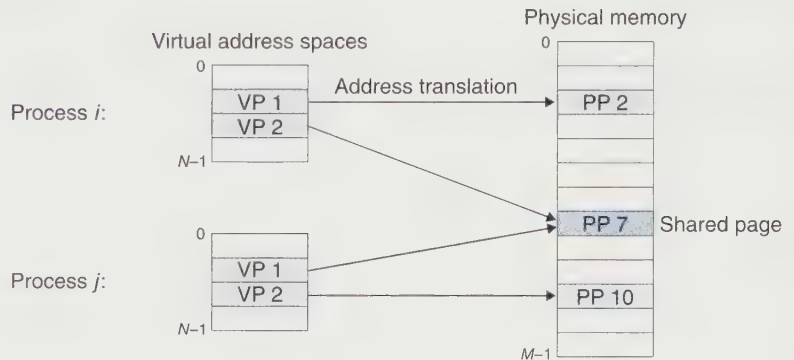
As long as our programs have good temporal locality, virtual memory systems work quite well. But of course, not all programs exhibit good temporal locality. If the working set size exceeds the size of physical memory, then the program can produce an unfortunate situation known as *thrashing*, where pages are swapped in and out continuously. Although virtual memory is usually efficient, if a program's performance slows to a crawl, the wise programmer will consider the possibility that it is thrashing.

Aside Counting page faults

You can monitor the number of page faults (and lots of other information) with the Linux `getrusage` function.

Figure 9.9

How VM provides processes with separate address spaces. The operating system maintains a separate page table for each process in the system.



9.4 VM as a Tool for Memory Management

In the last section, we saw how virtual memory provides a mechanism for using the DRAM to cache pages from a typically larger virtual address space. Interestingly, some early systems such as the DEC PDP-11/70 supported a virtual address space that was *smaller* than the available physical memory. Yet virtual memory was still a useful mechanism because it greatly simplified memory management and provided a natural way to protect memory.

Thus far, we have assumed a single page table that maps a single virtual address space to the physical address space. In fact, operating systems provide a separate page table, and thus a separate virtual address space, for each process. Figure 9.9 shows the basic idea. In the example, the page table for process *i* maps VP 1 to PP 2 and VP 2 to PP 7. Similarly, the page table for process *j* maps VP 1 to PP 7 and VP 2 to PP 10. Notice that multiple virtual pages can be mapped to the same shared physical page.

The combination of demand paging and separate virtual address spaces has a profound impact on the way that memory is used and managed in a system. In particular, VM simplifies linking and loading, the sharing of code and data, and allocating memory to applications.

- *Simplifying linking.* A separate address space allows each process to use the same basic format for its memory image, regardless of where the code and data actually reside in physical memory. For example, as we saw in Figure 8.13, every process on a given Linux system has a similar memory format. For 64-bit address spaces, the code segment *always* starts at virtual address `0x400000`. The data segment follows the code segment after a suitable alignment gap. The stack occupies the highest portion of the user process address space and

grows downward. Such uniformity greatly simplifies the design and implementation of linkers, allowing them to produce fully linked executables that are independent of the ultimate location of the code and data in physical memory.

- *Simplifying loading.* Virtual memory also makes it easy to load executable and shared object files into memory. To load the `.text` and `.data` sections of an object file into a newly created process, the Linux loader allocates virtual pages for the code and data segments, marks them as invalid (i.e., not cached), and points their page table entries to the appropriate locations in the object file. The interesting point is that the loader never actually copies any data from disk into memory. The data are paged in automatically and on demand by the virtual memory system the first time each page is referenced, either by the CPU when it fetches an instruction or by an executing instruction when it references a memory location.

This notion of mapping a set of contiguous virtual pages to an arbitrary location in an arbitrary file is known as *memory mapping*. Linux provides a system call called `mmap` that allows application programs to do their own memory mapping. We will describe application-level memory mapping in more detail in Section 9.8.

- *Simplifying sharing.* Separate address spaces provide the operating system with a consistent mechanism for managing sharing between user processes and the operating system itself. In general, each process has its own private code, data, heap, and stack areas that are not shared with any other process. In this case, the operating system creates page tables that map the corresponding virtual pages to disjoint physical pages.

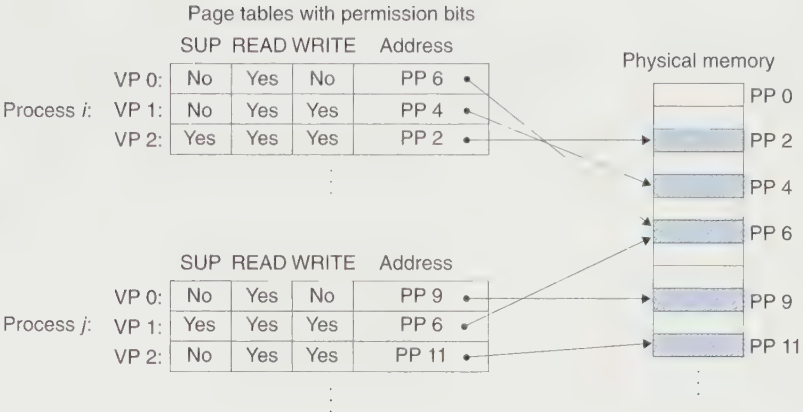
However, in some instances it is desirable for processes to share code and data. For example, every process must call the same operating system kernel code, and every C program makes calls to routines in the standard C library such as `printf`. Rather than including separate copies of the kernel and standard C library in each process, the operating system can arrange for multiple processes to share a single copy of this code by mapping the appropriate virtual pages in different processes to the same physical pages, as we saw in Figure 9.9.

- *Simplifying memory allocation.* Virtual memory provides a simple mechanism for allocating additional memory to user processes. When a program running in a user process requests additional heap space (e.g., as a result of calling `malloc`), the operating system allocates an appropriate number, say, k , of contiguous virtual memory pages, and maps them to k arbitrary physical pages located anywhere in physical memory. Because of the way page tables work, there is no need for the operating system to locate k contiguous pages of physical memory. The pages can be scattered randomly in physical memory.

9.5 VM as a Tool for Memory Protection

Any modern computer system must provide the means for the operating system to control access to the memory system. A user process should not be allowed

Figure 9.10
Using VM to provide
page-level memory
protection.



to modify its read-only code section. Nor should it be allowed to read or modify any of the code and data structures in the kernel. It should not be allowed to read or write the private memory of other processes, and it should not be allowed to modify any virtual pages that are shared with other processes, unless all parties explicitly allow it (via calls to explicit interprocess communication system calls).

As we have seen, providing separate virtual address spaces makes it easy to isolate the private memories of different processes. But the address translation mechanism can be extended in a natural way to provide even finer access control. Since the address translation hardware reads a PTE each time the CPU generates an address, it is straightforward to control access to the contents of a virtual page by adding some additional permission bits to the PTE. Figure 9.10 shows the general idea.

In this example, we have added three permission bits to each PTE. The SUP bit indicates whether processes must be running in kernel (supervisor) mode to access the page. Processes running in kernel mode can access any page, but processes running in user mode are only allowed to access pages for which SUP is 0. The READ and WRITE bits control read and write access to the page. For example, if process *i* is running in user mode, then it has permission to read VP 0 and to read or write VP 1. However, it is not allowed to access VP 2.

If an instruction violates these permissions, then the CPU triggers a general protection fault that transfers control to an exception handler in the kernel, which sends a SIGSEGV signal to the offending process. Linux shells typically report this exception as a “segmentation fault.”

9.6 Address Translation

This section covers the basics of address translation. Our aim is to give you an appreciation of the hardware’s role in supporting virtual memory, with enough detail so that you can work through some concrete examples by hand. However, keep in mind that we are omitting a number of details, especially related to timing,

Symbol	Description
Basic parameters	
$N = 2^n$	Number of addresses in virtual address space
$M = 2^m$	Number of addresses in physical address space
$P = 2^p$	Page size (bytes)
Components of a virtual address (VA)	
VPO	Virtual page offset (bytes)
VPN	Virtual page number
TLBI	TLB index
TLBT	TLB tag
Components of a physical address (PA)	
PPO	Physical page offset (bytes)
PPN	Physical page number
CO	Byte offset within cache block
CI	Cache index
CT	Cache tag

Figure 9.11 Summary of address translation symbols.

that are important to hardware designers but are beyond our scope. For your reference, Figure 9.11 summarizes the symbols that we will be using throughout this section.

Formally, address translation is a mapping between the elements of an N -element virtual address space (VAS) and an M -element physical address space (PAS),

$$\text{MAP: VAS} \rightarrow \text{PAS} \cup \emptyset$$

where

$$\text{MAP}(A) = \begin{cases} A' & \text{if data at virtual addr. } A \text{ are present at physical addr. } A' \text{ in PAS} \\ \emptyset & \text{if data at virtual addr. } A \text{ are not present in physical memory} \end{cases}$$

Figure 9.12 shows how the MMU uses the page table to perform this mapping. A control register in the CPU, the *page table base register (PTBR)* points to the current page table. The n -bit virtual address has two components: a p -bit *virtual page offset (VPO)* and an $(n - p)$ -bit *virtual page number (VPN)*. The MMU uses the VPN to select the appropriate PTE. For example, VPN 0 selects PTE 0, VPN 1 selects PTE 1, and so on. The corresponding physical address is the concatenation of the *physical page number (PPN)* from the page table entry and the VPO from the virtual address. Notice that since the physical and virtual pages are both P bytes, the *physical page offset (PPO)* is identical to the VPO.

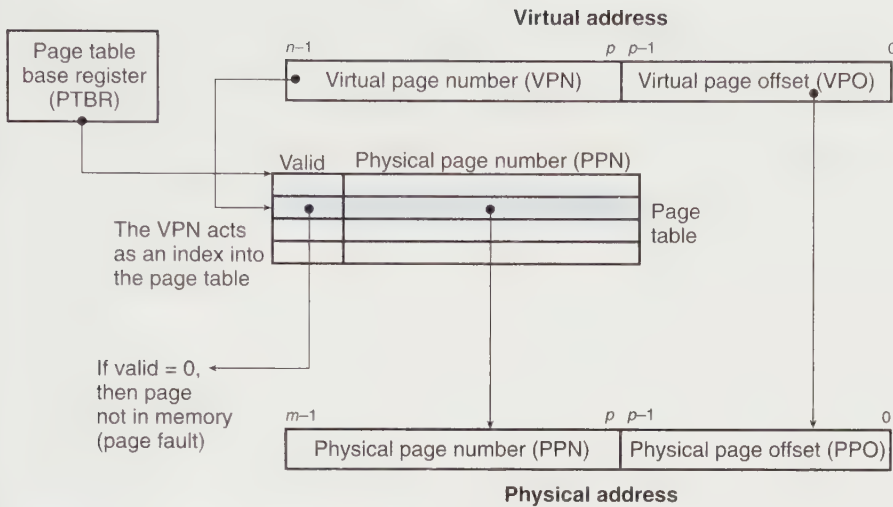


Figure 9.12 Address translation with a page table.

Figure 9.13(a) shows the steps that the CPU hardware performs when there is a page hit.

- Step 1.* The processor generates a virtual address and sends it to the MMU.
- Step 2.* The MMU generates the PTE address and requests it from the cache/main memory.
- Step 3.* The cache/main memory returns the PTE to the MMU.
- Step 4.* The MMU constructs the physical address and sends it to the cache/main memory.
- Step 5.* The cache/main memory returns the requested data word to the processor.

Unlike a page hit, which is handled entirely by hardware, handling a page fault requires cooperation between hardware and the operating system kernel (Figure 9.13(b)).

- Steps 1 to 3.* The same as steps 1 to 3 in Figure 9.13(a).
- Step 4.* The valid bit in the PTE is zero, so the MMU triggers an exception, which transfers control in the CPU to a page fault exception handler in the operating system kernel.
- Step 5.* The fault handler identifies a victim page in physical memory, and if that page has been modified, pages it out to disk.
- Step 6.* The fault handler pages in the new page and updates the PTE in memory.

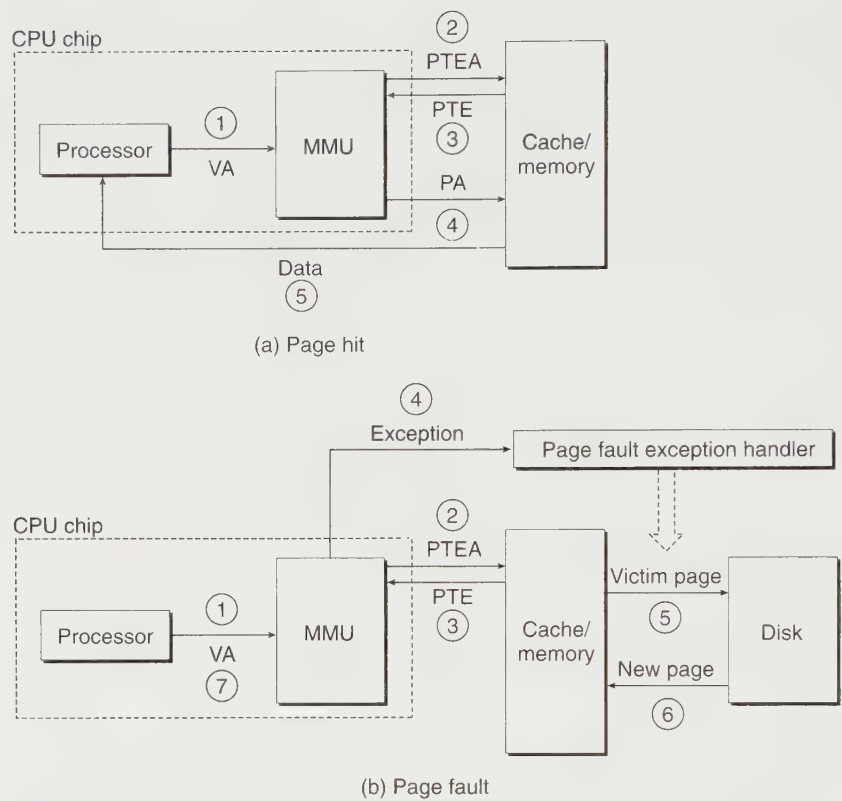


Figure 9.13 Operational view of page hits and page faults. VA: virtual address. PTEA: page table entry address. PTE: page table entry. PA: physical address.

Step 7. The fault handler returns to the original process, causing the faulting instruction to be restarted. The CPU resends the offending virtual address to the MMU. Because the virtual page is now cached in physical memory, there is a hit, and after the MMU performs the steps in Figure 9.13(a), the main memory returns the requested word to the processor.

Practice Problem 9.3 (solution page 881)

Given a 32-bit virtual address space and a 24-bit physical address, determine the number of bits in the VPN, VPO, PPN, and PPO for the following page sizes P :

P	Number of			
	VPN bits	VPO bits	PPN bits	PPO bits
1 KB				
2 KB				
4 KB				
8 KB				

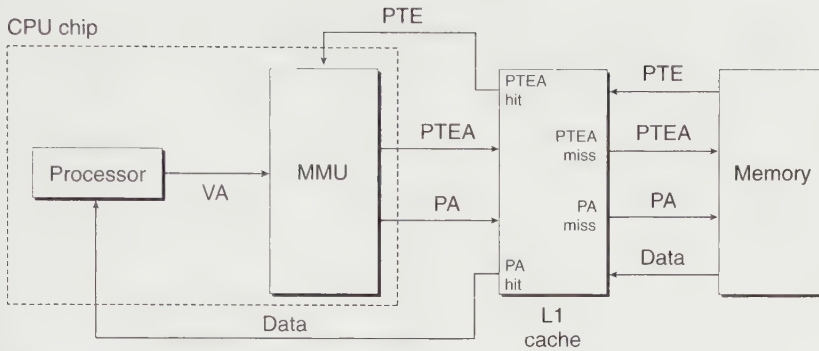


Figure 9.14 Integrating VM with a physically addressed cache. VA: virtual address. PTEA: page table entry address. PTE: page table entry. PA: physical address.

9.6.1 Integrating Caches and VM

In any system that uses both virtual memory and SRAM caches, there is the issue of whether to use virtual or physical addresses to access the SRAM cache. Although a detailed discussion of the trade-offs is beyond our scope here, most systems opt for physical addressing. With physical addressing, it is straightforward for multiple processes to have blocks in the cache at the same time and to share blocks from the same virtual pages. Further, the cache does not have to deal with protection issues, because access rights are checked as part of the address translation process.

Figure 9.14 shows how a physically addressed cache might be integrated with virtual memory. The main idea is that the address translation occurs before the cache lookup. Notice that page table entries can be cached, just like any other data words.

9.6.2 Speeding Up Address Translation with a TLB

As we have seen, every time the CPU generates a virtual address, the MMU must refer to a PTE in order to translate the virtual address into a physical address. In the worst case, this requires an additional fetch from memory, at a cost of tens to hundreds of cycles. If the PTE happens to be cached in L1, then the cost goes down to a handful of cycles. However, many systems try to eliminate even this cost by including a small cache of PTEs in the MMU called a *translation lookaside buffer (TLB)*.

A TLB is a small, virtually addressed cache where each line holds a block consisting of a single PTE. A TLB usually has a high degree of associativity. As shown in Figure 9.15, the index and tag fields that are used for set selection and line matching are extracted from the virtual page number in the virtual address. If the TLB has $T = 2^t$ sets, then the *TLB index (TLBI)* consists of the t least significant bits of the VPN, and the *TLB tag (TLBT)* consists of the remaining bits in the VPN.

Figure 9.15
Components of a virtual address that are used to access the TLB.

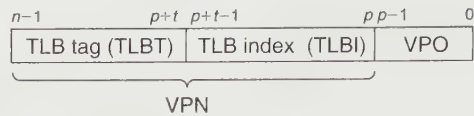


Figure 9.16
Operational view of a TLB hit and miss.

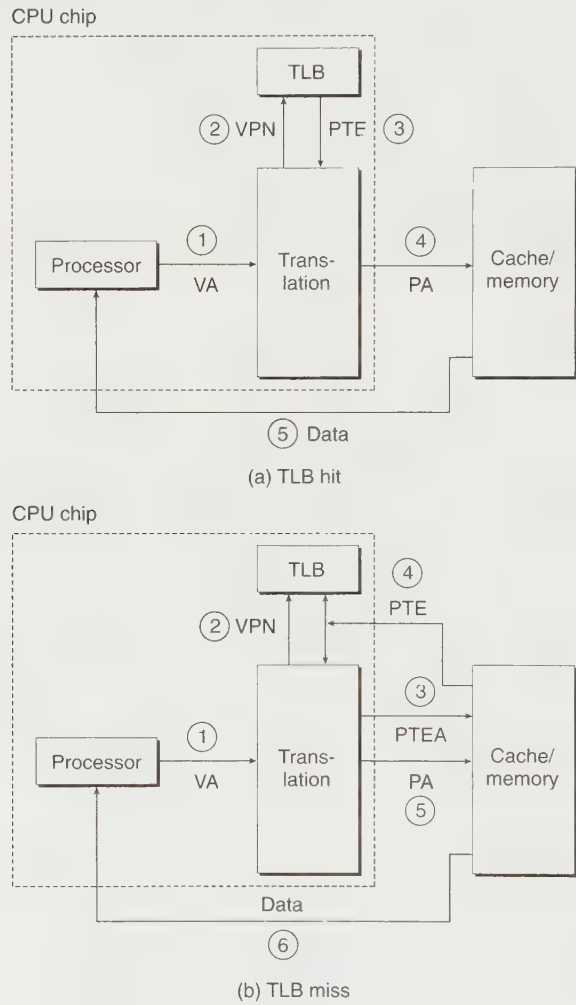


Figure 9.16(a) shows the steps involved when there is a TLB hit (the usual case). The key point here is that all of the address translation steps are performed inside the on-chip MMU and thus are fast.

- Step 1. The CPU generates a virtual address.
- Steps 2 and 3. The MMU fetches the appropriate PTE from the TLB.

Step 4. The MMU translates the virtual address to a physical address and sends it to the cache/main memory.

Step 5. The cache/main memory returns the requested data word to the CPU.

When there is a TLB miss, then the MMU must fetch the PTE from the L1 cache, as shown in Figure 9.16(b). The newly fetched PTE is stored in the TLB, possibly overwriting an existing entry.

9.6.3 Multi-Level Page Tables

Thus far, we have assumed that the system uses a single page table to do address translation. But if we had a 32-bit address space, 4 KB pages, and a 4-byte PTE, then we would need a 4 MB page table resident in memory at all times, even if the application referenced only a small chunk of the virtual address space. The problem is compounded for systems with 64-bit address spaces.

The common approach for compacting the page table is to use a hierarchy of page tables instead. The idea is easiest to understand with a concrete example. Consider a 32-bit virtual address space partitioned into 4 KB pages, with page table entries that are 4 bytes each. Suppose also that at this point in time the virtual address space has the following form: The first 2 K pages of memory are allocated for code and data, the next 6 K pages are unallocated, the next 1,023 pages are also unallocated, and the next page is allocated for the user stack. Figure 9.17 shows how we might construct a two-level page table hierarchy for this virtual address space.

Each PTE in the level 1 table is responsible for mapping a 4 MB chunk of the virtual address space, where each chunk consists of 1,024 contiguous pages. For example, PTE 0 maps the first chunk, PTE 1 the next chunk, and so on. Given that the address space is 4 GB, 1,024 PTEs are sufficient to cover the entire space.

If every page in chunk i is unallocated, then level 1 PTE i is null. For example, in Figure 9.17, chunks 2–7 are unallocated. However, if at least one page in chunk i is allocated, then level 1 PTE i points to the base of a level 2 page table. For example, in Figure 9.17, all or portions of chunks 0, 1, and 8 are allocated, so their level 1 PTEs point to level 2 page tables.

Each PTE in a level 2 page table is responsible for mapping a 4-KB page of virtual memory, just as before when we looked at single-level page tables. Notice that with 4-byte PTEs, each level 1 and level 2 page table is 4 kilobytes, which conveniently is the same size as a page.

This scheme reduces memory requirements in two ways. First, if a PTE in the level 1 table is null, then the corresponding level 2 page table does not even have to exist. This represents a significant potential savings, since most of the 4 GB virtual address space for a typical program is unallocated. Second, only the level 1 table needs to be in main memory at all times. The level 2 page tables can be created and paged in and out by the VM system as they are needed, which reduces pressure on main memory. Only the most heavily used level 2 page tables need to be cached in main memory.

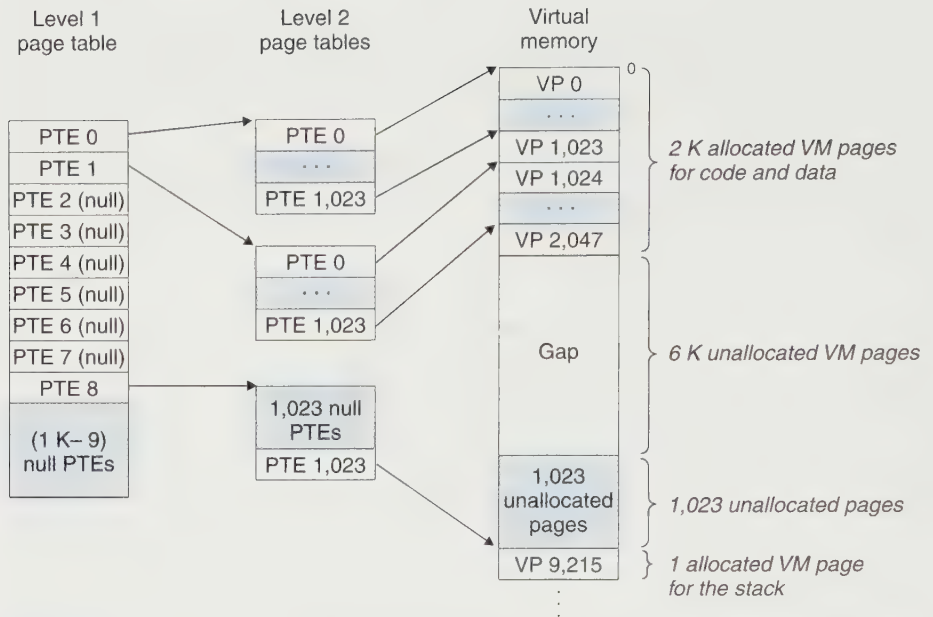


Figure 9.17 A two-level page table hierarchy. Notice that addresses increase from top to bottom.

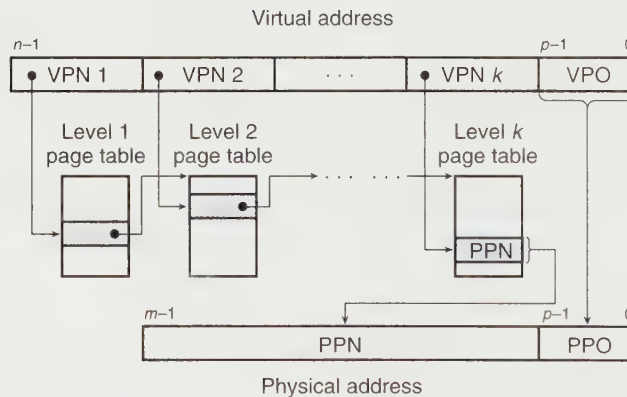


Figure 9.18 Address translation with a k -level page table.

Figure 9.18 summarizes address translation with a k -level page table hierarchy. The virtual address is partitioned into k VPNs and a VPO. Each VPN i , $1 \leq i \leq k$, is an index into a page table at level i . Each PTE in a level j table, $1 \leq j \leq k-1$, points to the base of some page table at level $j+1$. Each PTE in a level k table contains either the PPN of some physical page or the address of a disk block. To construct the physical address, the MMU must access k PTEs before it can

determine the PPN. As with a single-level hierarchy, the PPO is identical to the VPO.

Accessing k PTEs may seem expensive and impractical at first glance. However, the TLB comes to the rescue here by caching PTEs from the page tables at the different levels. In practice, address translation with multi-level page tables is not significantly slower than with single-level page tables.

9.6.4 Putting It Together: End-to-End Address Translation

In this section, we put it all together with a concrete example of end-to-end address translation on a small system with a TLB and L1 d-cache. To keep things manageable, we make the following assumptions:

- The memory is byte addressable.
- Memory accesses are to *1-byte words* (not 4-byte words).
- Virtual addresses are 14 bits wide ($n = 14$).
- Physical addresses are 12 bits wide ($m = 12$).
- The page size is 64 bytes ($P = 64$).
- The TLB is 4-way set associative with 16 total entries.
- The L1 d-cache is physically addressed and direct mapped, with a 4-byte line size and 16 total sets.

Figure 9.19 shows the formats of the virtual and physical addresses. Since each page is $2^6 = 64$ bytes, the low-order 6 bits of the virtual and physical addresses serve as the VPO and PPO, respectively. The high-order 8 bits of the virtual address serve as the VPN. The high-order 6 bits of the physical address serve as the PPN.

Figure 9.20 shows a snapshot of our little memory system, including the TLB (Figure 9.20(a)), a portion of the page table (Figure 9.20(b)), and the L1 cache (Figure 9.20(c)). Above the figures of the TLB and cache, we have also shown how the bits of the virtual and physical addresses are partitioned by the hardware as it accesses these devices.

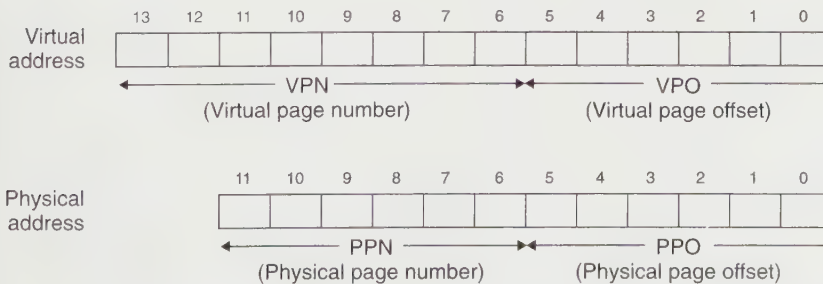
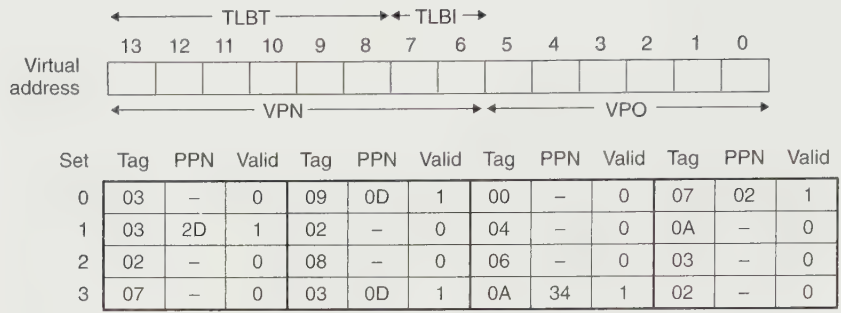


Figure 9.19 Addressing for small memory system. Assume 14-bit virtual addresses ($n = 14$), 12-bit physical addresses ($m = 12$), and 64-byte pages ($P = 64$).



(a) TLB: 4 sets, 16 entries, 4-way set associative

Diagram illustrating the Page Table structure.

VPN	PPN	Valid	VPN	PPN	Valid
00	28	1	08	13	1
01	—	0	09	17	1
02	33	1	0A	09	1
03	02	1	0B	—	0
04	—	0	0C	—	0
05	16	1	0D	2D	1
06	—	0	0E	11	1
07	—	0	0F	0D	1

(b) Page table: Only the first 16 PTEs are shown

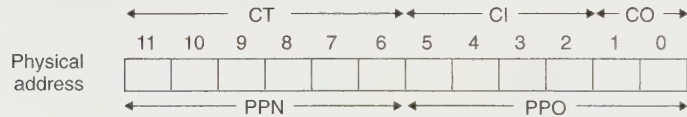


Diagram illustrating the Cache structure (16 sets, 4-byte blocks, direct mapped).

Idx	Tag	Valid	Blk 0	Blk 1	Blk 2	Blk 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	08
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

(c) Cache: 16 sets, 4-byte blocks, direct mapped

Figure 9.20 TLB, page table, and cache for small memory system. All values in the TLB, page table, and cache are in hexadecimal notation.

TLB. The TLB is virtually addressed using the bits of the VPN. Since the TLB has four sets, the 2 low-order bits of the VPN serve as the set index (TLBI). The remaining 6 high-order bits serve as the tag (TLBT) that distinguishes the different VPNs that might map to the same TLB set.

Page table. The page table is a single-level design with a total of $2^8 = 256$ page table entries (PTEs). However, we are only interested in the first 16 of these. For convenience, we have labeled each PTE with the VPN that indexes it; but keep in mind that these VPNs are not part of the page table and not stored in memory. Also, notice that the PPN of each invalid PTE is denoted with a dash to reinforce the idea that whatever bit values might happen to be stored there are not meaningful.

Cache. The direct-mapped cache is addressed by the fields in the physical address. Since each block is 4 bytes, the low-order 2 bits of the physical address serve as the block offset (CO). Since there are 16 sets, the next 4 bits serve as the set index (CI). The remaining 6 bits serve as the tag (CT).

Given this initial setup, let's see what happens when the CPU executes a load instruction that reads the byte at address 0x03d4. (Recall that our hypothetical CPU reads 1-byte words rather than 4-byte words.) To begin this kind of manual simulation, we find it helpful to write down the bits in the virtual address, identify the various fields we will need, and determine their hex values. The hardware performs a similar task when it decodes the address.

Bit position	TLBT						TLBI							
	0x03						0x03							
	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VA = 0x03d4	0	0	0	0	1	1	1	1	0	1	0	1	0	0
VPN									VPO					
0x0f									0x14					

To begin, the MMU extracts the VPN (0x0F) from the virtual address and checks with the TLB to see if it has cached a copy of PTE 0x0F from some previous memory reference. The TLB extracts the TLB index (0x03) and the TLB tag (0x3) from the VPN, hits on a valid match in the second entry of set 0x3, and returns the cached PPN (0x0D) to the MMU.

If the TLB had missed, then the MMU would need to fetch the PTE from main memory. However, in this case, we got lucky and had a TLB hit. The MMU now has everything it needs to form the physical address. It does this by concatenating the PPN (0x0D) from the PTE with the VPO (0x14) from the virtual address, which forms the physical address (0x354).

Next, the MMU sends the physical address to the cache, which extracts the cache offset CO (0x0), the cache set index CI (0x5), and the cache tag CT (0x0D) from the physical address.

	CT						CI				CO	
	0x0d						0x05				0x0	
Bit position	11	10	9	8	7	6	5	4	3	2	1	0
PA = 0x354	0	0	1	1	0	1	0	1	0	1	0	0
	PPN						PPO					
	0x0d						0x14					

Since the tag in set 0x5 matches CT, the cache detects a hit, reads out the data byte (0x36) at offset CO, and returns it to the MMU, which then passes it back to the CPU.

Other paths through the translation process are also possible. For example, if the TLB misses, then the MMU must fetch the PPN from a PTE in the page table. If the resulting PTE is invalid, then there is a page fault and the kernel must page in the appropriate page and rerun the load instruction. Another possibility is that the PTE is valid, but the necessary memory block misses in the cache.

Practice Problem 9.4 (solution page 881)

Show how the example memory system in Section 9.6.4 translates a virtual address into a physical address and accesses the cache. For the given virtual address, indicate the TLB entry accessed, physical address, and cache byte value returned. Indicate whether the TLB misses, whether a page fault occurs, and whether a cache miss occurs. If there is a cache miss, enter “—” for “Cache byte returned.” If there is a page fault, enter “—” for “PPN” and leave parts C and D blank.

Virtual address: 0x03d7

A. Virtual address format

[illegible]

B. Address translation

Parameter	Value
VPN	
TLB index	
TLB tag	
TLB hit? (Y/N)	
Page fault? (Y/N)	
PPN	

C. Physical address format

[illegible]

D. Physical memory reference

Parameter	Value
Byte offset	
Cache index	
Cache tag	
Cache hit? (Y/N)	
Cache byte returned	

9.7 Case Study: The Intel Core i7/Linux Memory System

We conclude our discussion of virtual memory mechanisms with a case study of a real system: an Intel Core i7 running Linux. Although the underlying Haswell microarchitecture allows for full 64-bit virtual and physical address spaces, the current Core i7 implementations (and those for the foreseeable future) support a 48-bit (256 TB) virtual address space and a 52-bit (4 PB) physical address space, along with a compatibility mode that supports 32-bit (4 GB) virtual and physical address spaces.

Figure 9.21 gives the highlights of the Core i7 memory system. The *processor package* (chip) includes four cores, a large L3 cache shared by all of the cores, and

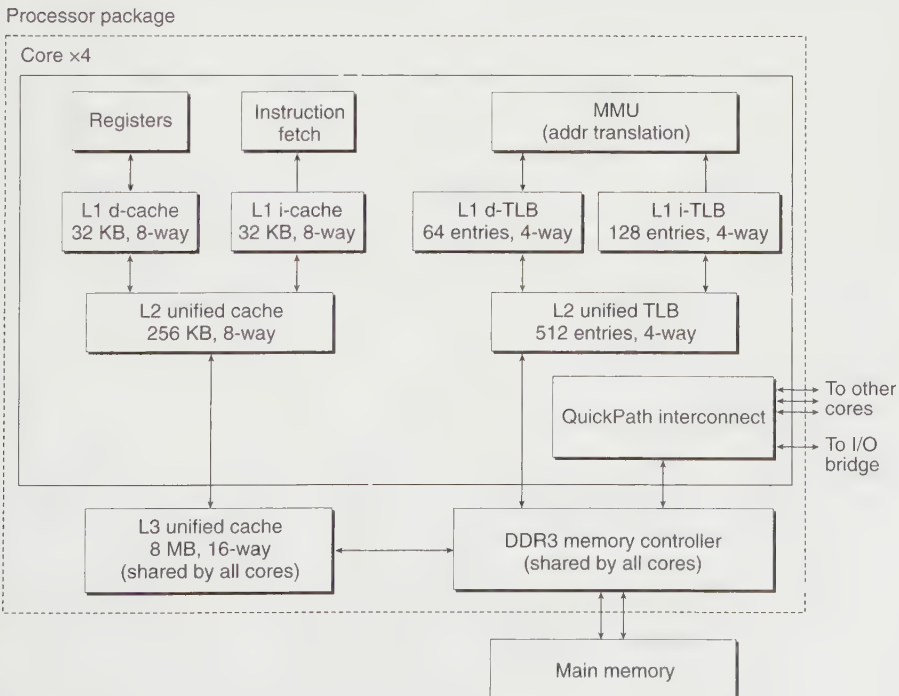


Figure 9.21 The Core i7 memory system.

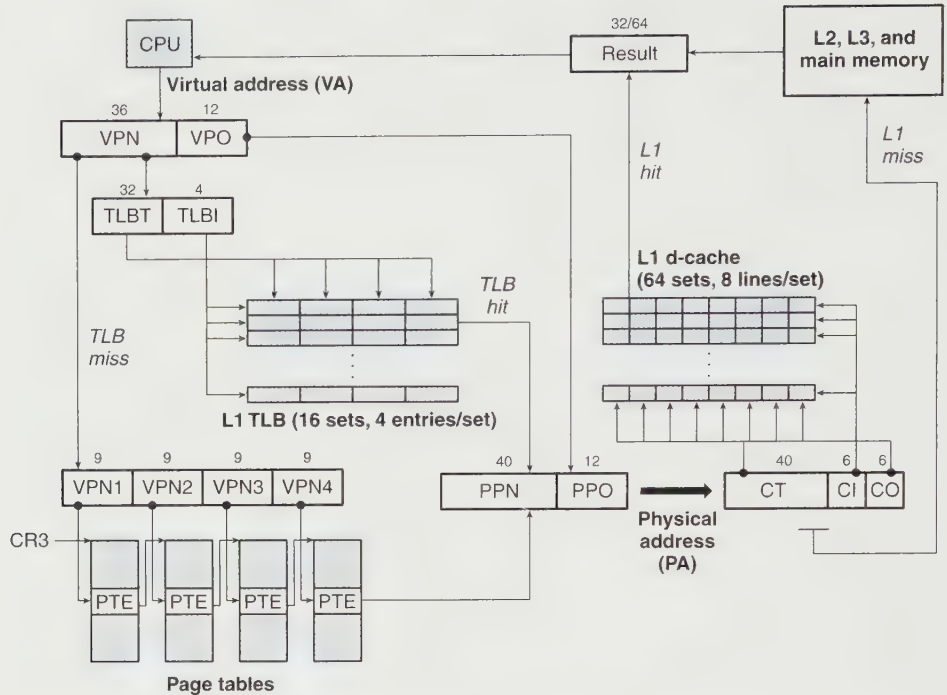
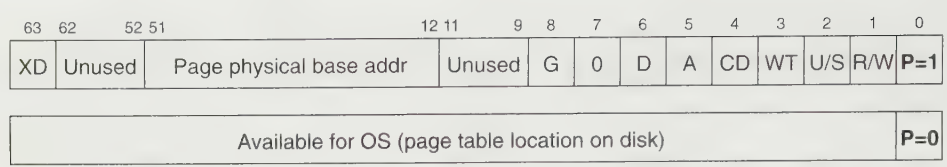


Figure 9.22 Summary of Core i7 address translation. For simplicity, the i-caches, i-TLB, and L2 unified TLB are not shown.

a DDR3 memory controller. Each core contains a hierarchy of TLBs, a hierarchy of data and instruction caches, and a set of fast point-to-point links, based on the QuickPath technology, for communicating directly with the other cores and the external I/O bridge. The TLBs are virtually addressed, and 4-way set associative. The L1, L2, and L3 caches are physically addressed, with a block size of 64 bytes. L1 and L2 are 8-way set associative, and L3 is 16-way set associative. The page size can be configured at start-up time as either 4 KB or 4 MB. Linux uses 4 KB pages.

9.7.1 Core i7 Address Translation

Figure 9.22 summarizes the entire Core i7 address translation process, from the time the CPU generates a virtual address until a data word arrives from memory. The Core i7 uses a four-level page table hierarchy. Each process has its own private page table hierarchy. When a Linux process is running, the page tables associated with allocated pages are all memory-resident, although the Core i7 architecture allows these page tables to be swapped in and out. The CR3 control register contains the physical address of the beginning of the level 1 (L1) page table. The value of CR3 is part of each process context, and is restored during each context switch.



Field	Description
P	Child page present in physical memory (1) or not (0).
R/W	Read-only or read/write access permission for child page.
U/S	User or supervisor mode (kernel mode) access permission for child page.
WT	Write-through or write-back cache policy for the child page.
CD	Cache disabled or enabled.
A	Reference bit (set by MMU on reads and writes, cleared by software).
D	Dirty bit (set by MMU on writes, cleared by software).
G	Global page (don't evict from TLB on task switch).
Base addr	40 most significant bits of physical base address of child page.
XD	Disable or enable instruction fetches from the child page.

Figure 9.24 Format of level 4 page table entries. Each entry references a 4 KB child page.

write back a victim page before it copies in a replacement page. The kernel can call a special kernel-mode instruction to clear the reference or dirty bits.

Figure 9.25 shows how the Core i7 MMU uses the four levels of page tables to translate a virtual address to a physical address. The 36-bit VPN is partitioned into four 9-bit chunks, each of which is used as an offset into a page table. The CR3 register contains the physical address of the L1 page table. VPN 1 provides an offset to an L1 PTE, which contains the base address of the L2 page table. VPN 2 provides an offset to an L2 PTE, and so on.

9.7.2 Linux Virtual Memory System

A virtual memory system requires close cooperation between the hardware and the kernel. Details vary from version to version, and a complete description is beyond our scope. Nonetheless, our aim in this section is to describe enough of the Linux virtual memory system to give you a sense of how a real operating system organizes virtual memory and how it handles page faults.

Linux maintains a separate virtual address space for each process of the form shown in Figure 9.26. We have seen this picture a number of times already, with its familiar code, data, heap, shared library, and stack segments. Now that we understand address translation, we can fill in some more details about the kernel virtual memory that lies above the user stack.

The kernel virtual memory contains the code and data structures in the kernel. Some regions of the kernel virtual memory are mapped to physical pages that

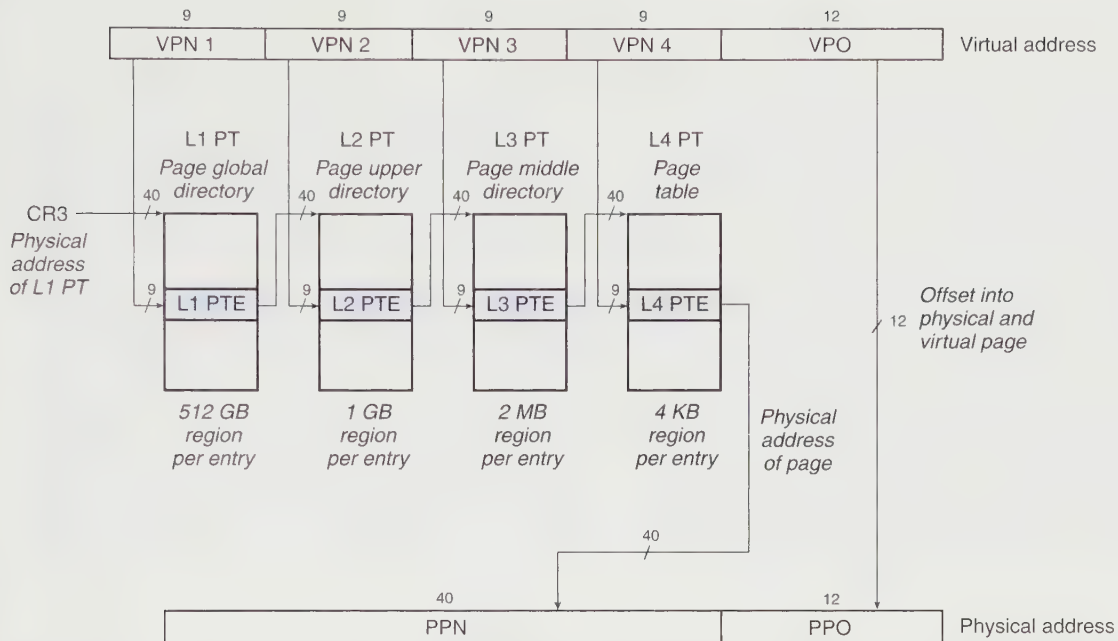
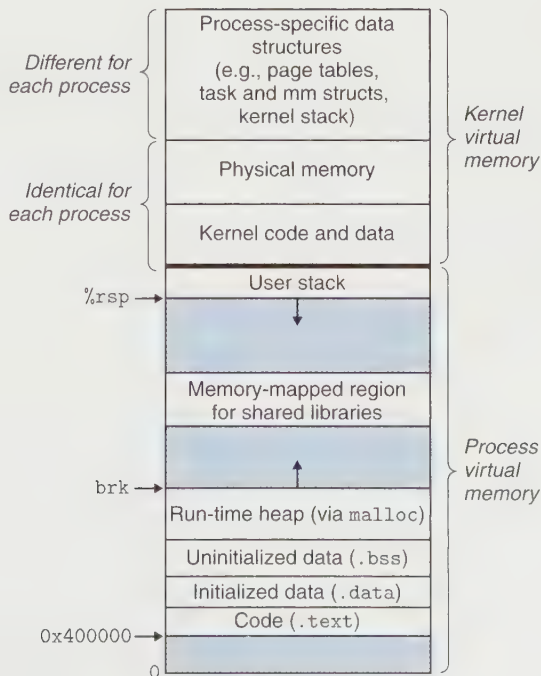


Figure 9.25 Core i7 page table translation. PT: page table; PTE: page table entry; VPN: virtual page number; VPO: virtual page offset; PPN: physical page number; PPO: physical page offset. The Linux names for the four levels of page tables are also shown.

Figure 9.26
The virtual memory of a Linux process.



Aside Optimizing address translation

In our discussion of address translation, we have described a sequential two-step process where the MMU (1) translates the virtual address to a physical address and then (2) passes the physical address to the L1 cache. However, real hardware implementations use a neat trick that allows these steps to be partially overlapped, thus speeding up accesses to the L1 cache. For example, a virtual address on a Core i7 with 4 KB pages has 12 bits of VPO, and these bits are identical to the 12 bits of PPO in the corresponding physical address. Since the 8-way set associative physically addressed L1 caches have 64 sets and 64-byte cache blocks, each physical address has 6 ($\log_2 64$) cache offset bits and 6 ($\log_2 64$) index bits. These 12 bits fit exactly in the 12-bit VPO of a virtual address, which is no accident! When the CPU needs a virtual address translated, it sends the VPN to the MMU and the VPO to the L1 cache. While the MMU is requesting a page table entry from the TLB, the L1 cache is busy using the VPO bits to find the appropriate set and read out the eight tags and corresponding data words in that set. When the MMU gets the PPN back from the TLB, the cache is ready to try to match the PPN to one of these eight tags.

are shared by all processes. For example, each process shares the kernel's code and global data structures. Interestingly, Linux also maps a set of contiguous virtual pages (equal in size to the total amount of DRAM in the system) to the corresponding set of contiguous physical pages. This provides the kernel with a convenient way to access any specific location in physical memory—for example, when it needs to access page tables or to perform memory-mapped I/O operations on devices that are mapped to particular physical memory locations.

Other regions of kernel virtual memory contain data that differ for each process. Examples include page tables, the stack that the kernel uses when it is executing code in the context of the process, and various data structures that keep track of the current organization of the virtual address space.

Linux Virtual Memory Areas

Linux organizes the virtual memory as a collection of *areas* (also called *segments*). An area is a contiguous chunk of existing (allocated) virtual memory whose pages are related in some way. For example, the code segment, data segment, heap, shared library segment, and user stack are all distinct areas. Each existing virtual page is contained in some area, and any virtual page that is not part of some area does not exist and cannot be referenced by the process. The notion of an area is important because it allows the virtual address space to have gaps. The kernel does not keep track of virtual pages that do not exist, and such pages do not consume any additional resources in memory, on disk, or in the kernel itself.

Figure 9.27 highlights the kernel data structures that keep track of the virtual memory areas in a process. The kernel maintains a distinct task structure (`task_struct` in the source code) for each process in the system. The elements of the task structure either contain or point to all of the information that the kernel needs to

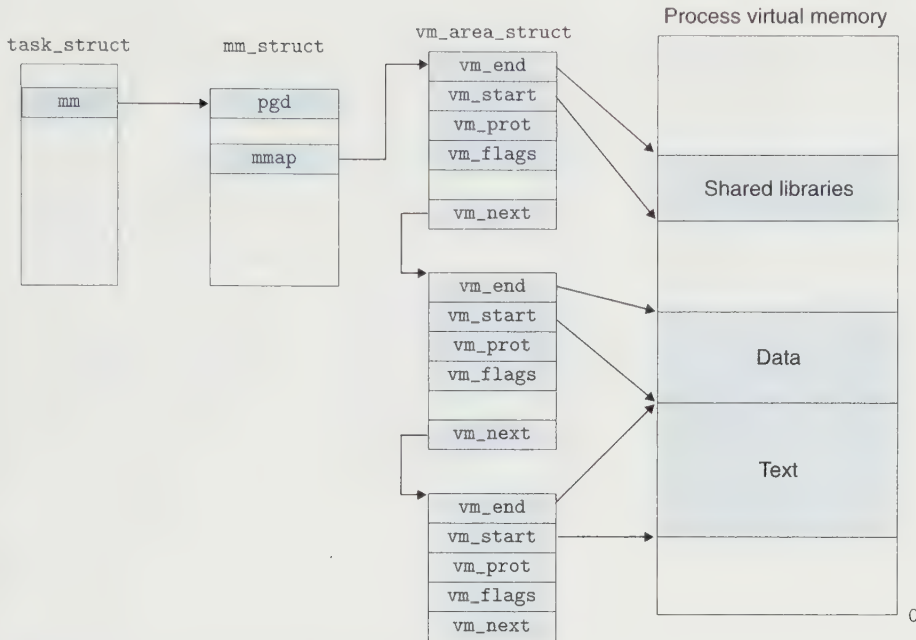


Figure 9.27 How Linux organizes virtual memory.

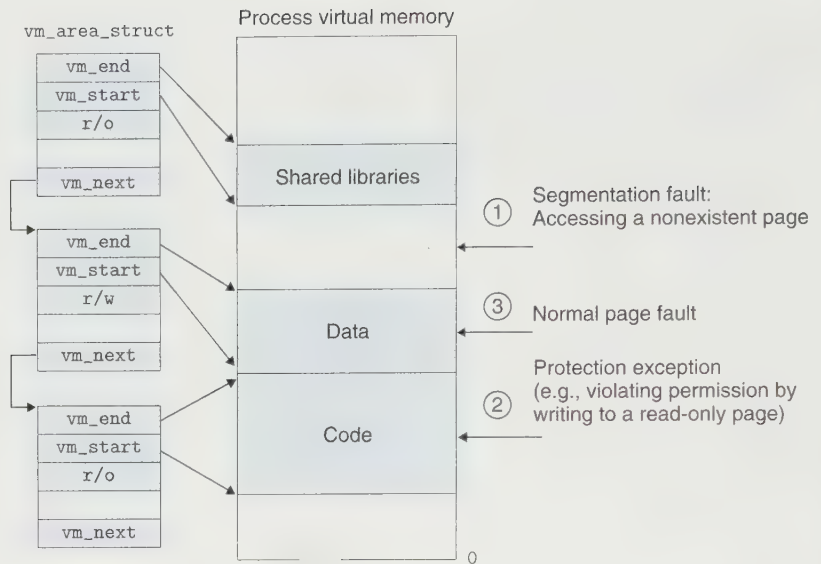
run the process (e.g., the PID, pointer to the user stack, name of the executable object file, and program counter).

One of the entries in the task structure points to an `mm_struct` that characterizes the current state of the virtual memory. The two fields of interest to us are `pgd`, which points to the base of the level 1 table (the page global directory), and `mmap`, which points to a list of `vm_area_structs` (area structs), each of which characterizes an area of the current virtual address space. When the kernel runs this process, it stores `pgd` in the CR3 control register.

For our purposes, the area struct for a particular area contains the following fields:

- `vm_start`. Points to the beginning of the area.
- `vm_end`. Points to the end of the area.
- `vm_prot`. Describes the read/write permissions for all of the pages contained in the area.
- `vm_flags`. Describes (among other things) whether the pages in the area are shared with other processes or private to this process.
- `vm_next`. Points to the next area struct in the list.

Figure 9.28
Linux page fault handling.



Linux Page Fault Exception Handling

Suppose the MMU triggers a page fault while trying to translate some virtual address *A*. The exception results in a transfer of control to the kernel's page fault handler, which then performs the following steps:

1. Is virtual address *A* legal? In other words, does *A* lie within an area defined by some area struct? To answer this question, the fault handler searches the list of area structs, comparing *A* with the `vm_start` and `vm_end` in each area struct. If the instruction is not legal, then the fault handler triggers a segmentation fault, which terminates the process. This situation is labeled "1" in Figure 9.28.

Because a process can create an arbitrary number of new virtual memory areas (using the `mmap` function described in the next section), a sequential search of the list of area structs might be very costly. So in practice, Linux superimposes a tree on the list, using some fields that we have not shown, and performs the search on this tree.

2. Is the attempted memory access legal? In other words, does the process have permission to read, write, or execute the pages in this area? For example, was the page fault the result of a store instruction trying to write to a read-only page in the code segment? Is the page fault the result of a process running in user mode that is attempting to read a word from kernel virtual memory? If the attempted access is not legal, then the fault handler triggers a protection exception, which terminates the process. This situation is labeled "2" in Figure 9.28.
3. At this point, the kernel knows that the page fault resulted from a legal operation on a legal virtual address. It handles the fault by selecting a victim page, swapping out the victim page if it is dirty, swapping in the new page,

and updating the page table. When the page fault handler returns, the CPU restarts the faulting instruction, which sends *A* to the MMU again. This time, the MMU translates *A* normally, without generating a page fault.

9.8 Memory Mapping

Linux initializes the contents of a virtual memory area by associating it with an *object* on disk, a process known as *memory mapping*. Areas can be mapped to one of two types of objects:

1. *Regular file in the Linux file system:* An area can be mapped to a contiguous section of a regular disk file, such as an executable object file. The file section is divided into page-size pieces, with each piece containing the initial contents of a virtual page. Because of demand paging, none of these virtual pages is actually swapped into physical memory until the CPU first *touches* the page (i.e., issues a virtual address that falls within that page's region of the address space). If the area is larger than the file section, then the area is padded with zeros.
2. *Anonymous file:* An area can also be mapped to an anonymous file, created by the kernel, that contains all binary zeros. The first time the CPU touches a virtual page in such an area, the kernel finds an appropriate victim page in physical memory, swaps out the victim page if it is dirty, overwrites the victim page with binary zeros, and updates the page table to mark the page as resident. Notice that no data are actually transferred between disk and memory. For this reason, pages in areas that are mapped to anonymous files are sometimes called *demand-zero pages*.

In either case, once a virtual page is initialized, it is swapped back and forth between a special *swap file* maintained by the kernel. The swap file is also known as the *swap space* or the *swap area*. An important point to realize is that at any point in time, the swap space bounds the total amount of virtual pages that can be allocated by the currently running processes.

9.8.1 Shared Objects Revisited

The idea of memory mapping resulted from a clever insight that if the virtual memory system could be integrated into the conventional file system, then it could provide a simple and efficient way to load programs and data into memory.

As we have seen, the process abstraction promises to provide each process with its own private virtual address space that is protected from errant writes or reads by other processes. However, many processes have identical read-only code areas. For example, each process that runs the Linux shell program `bash` has the same code area. Further, many programs need to access identical copies of read-only run-time library code. For example, every C program requires functions from the standard C library such as `printf`. It would be extremely wasteful for each process to keep duplicate copies of these commonly used codes in physical

memory. Fortunately, memory mapping provides us with a clean mechanism for controlling how objects are shared by multiple processes.

An object can be mapped into an area of virtual memory as either a *shared object* or a *private object*. If a process maps a shared object into an area of its virtual address space, then any writes that the process makes to that area are visible to any other processes that have also mapped the shared object into their virtual memory. Further, the changes are also reflected in the original object on disk.

Changes made to an area mapped to a private object, on the other hand, are not visible to other processes, and any writes that the process makes to the area are *not* reflected back to the object on disk. A virtual memory area into which a shared object is mapped is often called a *shared area*. Similarly for a *private area*.

Suppose that process 1 maps a shared object into an area of its virtual memory, as shown in Figure 9.29(a). Now suppose that process 2 maps the same shared ob-

Figure 9.29

A shared object. (a) After process 1 maps the shared object. (b) After process 2 maps the same shared object. (Note that the physical pages are not necessarily contiguous.)

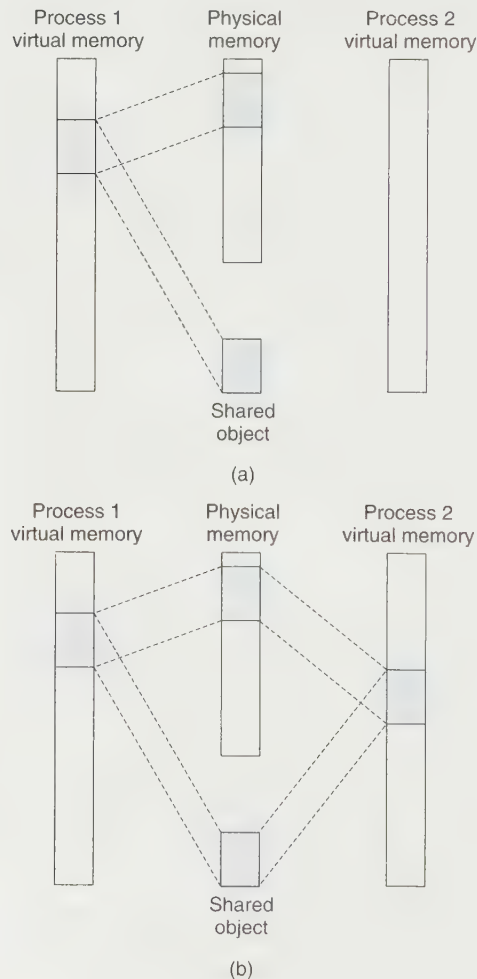
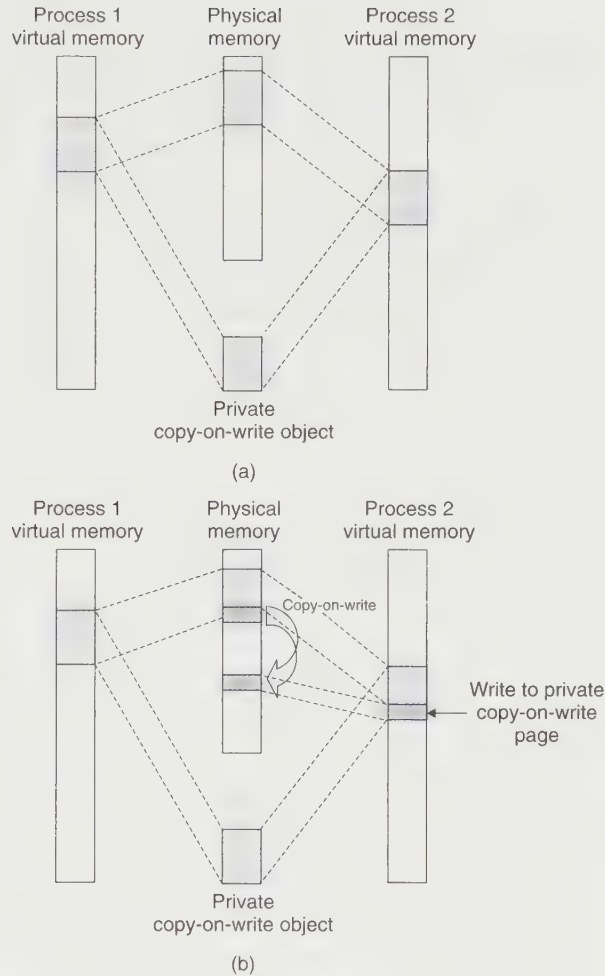


Figure 9.30

A private copy-on-write object. (a) After both processes have mapped the private copy-on-write object. (b) After process 2 writes to a page in the private area.



ject into its address space (not necessarily at the same virtual address as process 1), as shown in Figure 9.29(b).

Since each object has a unique filename, the kernel can quickly determine that process 1 has already mapped this object and can point the page table entries in process 2 to the appropriate physical pages. The key point is that only a single copy of the shared object needs to be stored in physical memory, even though the object is mapped into multiple shared areas. For convenience, we have shown the physical pages as being contiguous, but of course this is not true in general.

Private objects are mapped into virtual memory using a clever technique known as *copy-on-write*. A private object begins life in exactly the same way as a shared object, with only one copy of the private object stored in physical memory. For example, Figure 9.30(a) shows a case where two processes have mapped a private object into different areas of their virtual memories but share the same

physical copy of the object. For each process that maps the private object, the page table entries for the corresponding private area are flagged as read-only, and the area struct is flagged as *private copy-on-write*. So long as neither process attempts to write to its respective private area, they continue to share a single copy of the object in physical memory. However, as soon as a process attempts to write to some page in the private area, the write triggers a protection fault.

When the fault handler notices that the protection exception was caused by the process trying to write to a page in a private copy-on-write area, it creates a new copy of the page in physical memory, updates the page table entry to point to the new copy, and then restores write permissions to the page, as shown in Figure 9.30(b). When the fault handler returns, the CPU re-executes the write, which now proceeds normally on the newly created page.

By deferring the copying of the pages in private objects until the last possible moment, copy-on-write makes the most efficient use of scarce physical memory.

9.8.2 The `fork` Function Revisited

Now that we understand virtual memory and memory mapping, we can get a clear idea of how the `fork` function creates a new process with its own independent virtual address space.

When the `fork` function is called by the *current process*, the kernel creates various data structures for the *new process* and assigns it a unique PID. To create the virtual memory for the new process, it creates exact copies of the current process's `mm_struct`, area structs, and page tables. It flags each page in both processes as read-only, and flags each area struct in both processes as private copy-on-write.

When the `fork` returns in the new process, the new process now has an exact copy of the virtual memory as it existed when the `fork` was called. When either of the processes performs any subsequent writes, the copy-on-write mechanism creates new pages, thus preserving the abstraction of a private address space for each process.

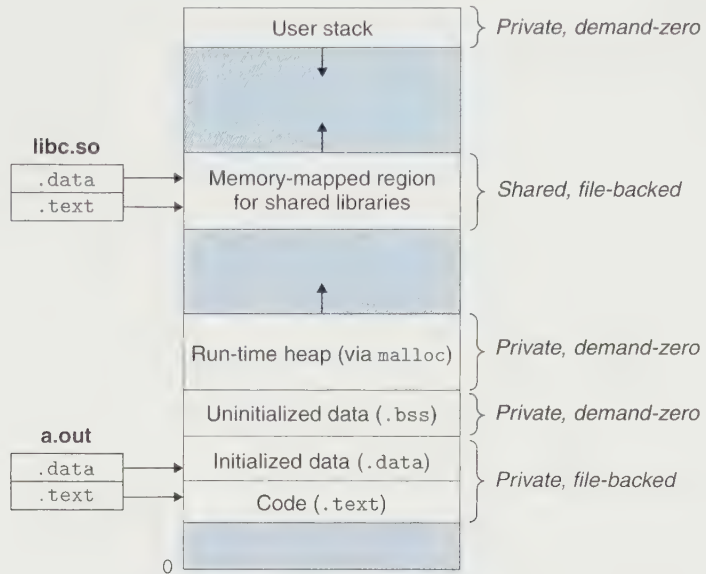
9.8.3 The `execve` Function Revisited

Virtual memory and memory mapping also play key roles in the process of loading programs into memory. Now that we understand these concepts, we can understand how the `execve` function really loads and executes programs. Suppose that the program running in the current process makes the following call:

```
execve("a.out", NULL, NULL);
```

As you learned in Chapter 8, the `execve` function loads and runs the program contained in the executable object file `a.out` within the current process, effectively replacing the current program with the `a.out` program. Loading and running `a.out` requires the following steps:

Figure 9.31
How the loader maps the
areas of the user address
space.



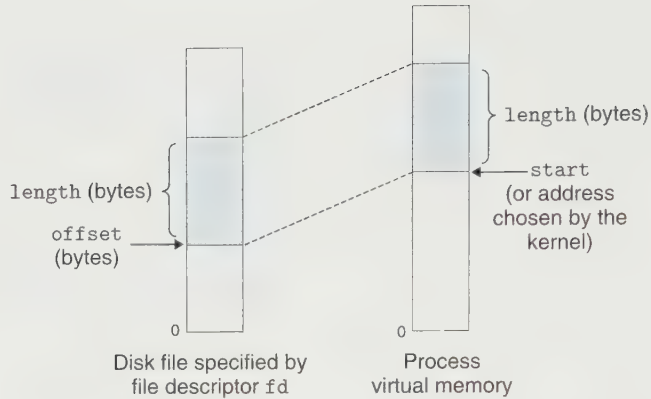
1. *Delete existing user areas.* Delete the existing area structs in the user portion of the current process's virtual address.
2. *Map private areas.* Create new area structs for the code, data, bss, and stack areas of the new program. All of these new areas are private copy-on-write. The code and data areas are mapped to the .text and .data sections of the a.out file. The bss area is demand-zero, mapped to an anonymous file whose size is contained in a.out. The stack and heap area are also demand-zero, initially of zero length. Figure 9.31 summarizes the different mappings of the private areas.
3. *Map shared areas.* If the a.out program was linked with shared objects, such as the standard C library libc.so, then these objects are dynamically linked into the program, and then mapped into the shared region of the user's virtual address space.
4. *Set the program counter (PC).* The last thing that execve does is to set the program counter in the current process's context to point to the entry point in the code area.

The next time this process is scheduled, it will begin execution from the entry point. Linux will swap in code and data pages as needed.

9.8.4 User-Level Memory Mapping with the mmap Function

Linux processes can use the mmap function to create new areas of virtual memory and to map objects into these areas.

Figure 9.32
Visual interpretation of
mmap arguments.



```
#include <unistd.h>
#include <sys/mman.h>

void *mmap(void *start, size_t length, int prot, int flags,
            int fd, off_t offset);

Returns: pointer to mapped area if OK, MAP_FAILED (-1) on error
```

The `mmap` function asks the kernel to create a new virtual memory area, preferably one that starts at address `start`, and to map a contiguous chunk of the object specified by file descriptor `fd` to the new area. The contiguous object chunk has a size of `length` bytes and starts at an offset of `offset` bytes from the beginning of the file. The `start` address is merely a hint, and is usually specified as `NULL`. For our purposes, we will always assume a `NULL` start address. Figure 9.32 depicts the meaning of these arguments.

The `prot` argument contains bits that describe the access permissions of the newly mapped virtual memory area (i.e., the `vm_prot` bits in the corresponding area struct).

PROT_EXEC. Pages in the area consist of instructions that may be executed by the CPU.

PROT_READ. Pages in the area may be read.

PROT_WRITE. Pages in the area may be written.

PROT_NONE. Pages in the area cannot be accessed.

The `flags` argument consists of bits that describe the type of the mapped object. If the `MAP_ANON` flag bit is set, then the backing store is an anonymous object and the corresponding virtual pages are demand-zero. `MAP_PRIVATE` indicates a private copy-on-write object, and `MAP_SHARED` indicates a shared object. For example,

```
bufp = Mmap(NULL, size, PROT_READ, MAP_PRIVATE|MAP_ANON, 0, 0);
```


asks the kernel to create a new read-only, private, demand-zero area of virtual memory containing `size` bytes. If the call is successful, then `bufp` contains the address of the new area.

The `munmap` function deletes regions of virtual memory:

```
#include <unistd.h>
#include <sys/mman.h>

int munmap(void *start, size_t length);
```

Returns: 0 if OK, -1 on error

The `munmap` function deletes the area starting at virtual address `start` and consisting of the next `length` bytes. Subsequent references to the deleted region result in segmentation faults.

Practice Problem 9.5 (solution page 882)

Write a C program `mmapcopy.c` that uses `mmap` to copy an arbitrary-size disk file to `stdout`. The name of the input file should be passed as a command-line argument.

9.9 Dynamic Memory Allocation

While it is certainly possible to use the low-level `mmap` and `munmap` functions to create and delete areas of virtual memory, C programmers typically find it more convenient and more portable to use a *dynamic memory allocator* when they need to acquire additional virtual memory at run time.

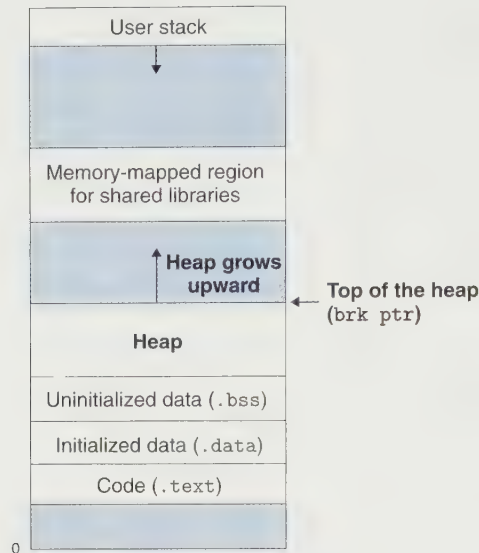
A dynamic memory allocator maintains an area of a process's virtual memory known as the *heap* (Figure 9.33). Details vary from system to system, but without loss of generality, we will assume that the heap is an area of demand-zero memory that begins immediately after the uninitialized data area and grows upward (toward higher addresses). For each process, the kernel maintains a variable `brk` (pronounced “break”) that points to the top of the heap.

An allocator maintains the heap as a collection of various-size *blocks*. Each block is a contiguous chunk of virtual memory that is either *allocated* or *free*. An allocated block has been explicitly reserved for use by the application. A free block is available to be allocated. A free block remains free until it is explicitly allocated by the application. An allocated block remains allocated until it is freed, either explicitly by the application or implicitly by the memory allocator itself.

Allocators come in two basic styles. Both styles require the application to explicitly allocate blocks. They differ about which entity is responsible for freeing allocated blocks.

- *Explicit allocators* require the application to explicitly free any allocated blocks. For example, the C standard library provides an explicit allocator called the `malloc` package. C programs allocate a block by calling the `malloc`

Figure 9.33
The heap.



function, and free a block by calling the `free` function. The `new` and `delete` calls in C++ are comparable.

- *Implicit allocators*, on the other hand, require the allocator to detect when an allocated block is no longer being used by the program and then free the block. Implicit allocators are also known as *garbage collectors*, and the process of automatically freeing unused allocated blocks is known as *garbage collection*. For example, higher-level languages such as Lisp, ML, and Java rely on garbage collection to free allocated blocks.

The remainder of this section discusses the design and implementation of explicit allocators. We will discuss implicit allocators in Section 9.10. For concreteness, our discussion focuses on allocators that manage heap memory. However, you should be aware that memory allocation is a general idea that arises in a variety of contexts. For example, applications that do intensive manipulation of graphs will often use the standard allocator to acquire a large block of virtual memory and then use an application-specific allocator to manage the memory within that block as the nodes of the graph are created and destroyed.

9.9.1 The `malloc` and `free` Functions

The C standard library provides an explicit allocator known as the `malloc` package. Programs allocate blocks from the heap by calling the `malloc` function.

```
#include <stdlib.h>

void *malloc(size_t size);
```

Returns: pointer to allocated block if OK, NULL on error

Aside How big is a word?

Recall from our discussion of machine code in Chapter 3 that Intel refers to 4-byte objects as *double words*. However, throughout this section, we will assume that *words* are 4-byte objects and that *double words* are 8-byte objects, which is consistent with conventional terminology.

The `malloc` function returns a pointer to a block of memory of at least `size` bytes that is suitably aligned for any kind of data object that might be contained in the block. In practice, the alignment depends on whether the code is compiled to run in 32-bit mode (`gcc -m32`) or 64-bit mode (the default). In 32-bit mode, `malloc` returns a block whose address is always a multiple of 8. In 64-bit mode, the address is always a multiple of 16.

If `malloc` encounters a problem (e.g., the program requests a block of memory that is larger than the available virtual memory), then it returns `NULL` and sets `errno`. `Malloc` does not initialize the memory it returns. Applications that want initialized dynamic memory can use `calloc`, a thin wrapper around the `malloc` function that initializes the allocated memory to zero. Applications that want to change the size of a previously allocated block can use the `realloc` function.

Dynamic memory allocators such as `malloc` can allocate or deallocate heap memory explicitly by using the `mmap` and `munmap` functions, or they can use the `sbrk` function:

```
#include <unistd.h>
```

```
void *sbrk(intptr_t incr);
```

Returns: old `brk` pointer on success, `-1` on error

The `sbrk` function grows or shrinks the heap by adding `incr` to the kernel's `brk` pointer. If successful, it returns the old value of `brk`, otherwise it returns `-1` and sets `errno` to `ENOMEM`. If `incr` is zero, then `sbrk` returns the current value of `brk`. Calling `sbrk` with a negative `incr` is legal but tricky because the return value (the old value of `brk`) points to `abs(incr)` bytes past the new top of the heap.

Programs free allocated heap blocks by calling the `free` function.

```
#include <stdlib.h>
```

```
void free(void *ptr);
```

Returns: nothing

The `ptr` argument must point to the beginning of an allocated block that was obtained from `malloc`, `calloc`, or `realloc`. If not, then the behavior of `free` is undefined. Even worse, since it returns nothing, `free` gives no indication to the application that something is wrong. As we shall see in Section 9.11, this can produce some baffling run-time errors.

Figure 9.34**Allocating and freeing blocks with malloc and free.**

Each square corresponds to a word. Each heavy rectangle corresponds to a block. Allocated blocks are shaded. Padded regions of allocated blocks are shaded with a darker blue. Free blocks are unshaded. Heap addresses increase from left to right.

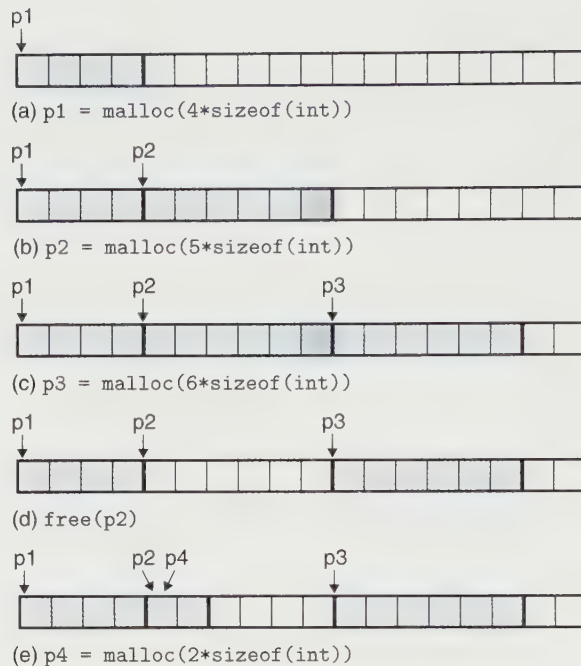


Figure 9.34 shows how an implementation of `malloc` and `free` might manage a (very) small heap of 16 words for a C program. Each box represents a 4-byte word. The heavy-lined rectangles correspond to allocated blocks (shaded) and free blocks (unshaded). Initially, the heap consists of a single 16-word double-word-aligned free block.¹

Figure 9.34(a). The program asks for a four-word block. `Malloc` responds by carving out a four-word block from the front of the free block and returning a pointer to the first word of the block.

Figure 9.34(b). The program requests a five-word block. `Malloc` responds by allocating a six-word block from the front of the free block. In this example, `malloc` pads the block with an extra word in order to keep the free block aligned on a double-word boundary.

Figure 9.34(c). The program requests a six-word block and `malloc` responds by carving out a six-word block from the free block.

Figure 9.34(d). The program frees the six-word block that was allocated in Figure 9.34(b). Notice that after the call to `free` returns, the pointer `p2`

1. Throughout this section, we will assume that the allocator returns blocks aligned to 8-byte double-word boundaries.

still points to the freed block. It is the responsibility of the application not to use `p2` again until it is reinitialized by a new call to `malloc`.

Figure 9.34(e). The program requests a two-word block. In this case, `malloc` allocates a portion of the block that was freed in the previous step and returns a pointer to this new block.

9.9.2 Why Dynamic Memory Allocation?

The most important reason that programs use dynamic memory allocation is that often they do not know the sizes of certain data structures until the program actually runs. For example, suppose we are asked to write a C program that reads a list of n ASCII integers, one integer per line, from `stdin` into a C array. The input consists of the integer n , followed by the n integers to be read and stored into the array. The simplest approach is to define the array statically with some hard-coded maximum array size:

```

1  #include "csapp.h"
2  #define MAXN 15213
3
4  int array[MAXN];
5
6  int main()
7  {
8      int i, n;
9
10     scanf("%d", &n);
11     if (n > MAXN)
12         app_error("Input file too big");
13     for (i = 0; i < n; i++)
14         scanf("%d", &array[i]);
15     exit(0);
16 }
```

Allocating arrays with hard-coded sizes like this is often a bad idea. The value of `MAXN` is arbitrary and has no relation to the actual amount of available virtual memory on the machine. Further, if the user of this program wanted to read a file that was larger than `MAXN`, the only recourse would be to recompile the program with a larger value of `MAXN`. While not a problem for this simple example, the presence of hard-coded array bounds can become a maintenance nightmare for large software products with millions of lines of code and numerous users.

A better approach is to allocate the array dynamically, at run time, after the value of n becomes known. With this approach, the maximum size of the array is limited only by the amount of available virtual memory.


```

1  #include "csapp.h"
2
3  int main()
4  {
5      int *array, i, n;
6
7      scanf("%d", &n);
8      array = (int *)Malloc(n * sizeof(int));
9      for (i = 0; i < n; i++)
10         scanf("%d", &array[i]);
11     free(array);
12     exit(0);
13 }

```

Dynamic memory allocation is a useful and important programming technique. However, in order to use allocators correctly and efficiently, programmers need to have an understanding of how they work. We will discuss some of the gruesome errors that can result from the improper use of allocators in Section 9.11.

9.9.3 Allocator Requirements and Goals

Explicit allocators must operate within some rather stringent constraints:

Handling arbitrary request sequences. An application can make an arbitrary sequence of allocate and free requests, subject to the constraint that each free request must correspond to a currently allocated block obtained from a previous allocate request. Thus, the allocator cannot make any assumptions about the ordering of allocate and free requests. For example, the allocator cannot assume that all allocate requests are accompanied by a matching free request, or that matching allocate and free requests are nested.

Making immediate responses to requests. The allocator must respond immediately to allocate requests. Thus, the allocator is not allowed to reorder or buffer requests in order to improve performance.

Using only the heap. In order for the allocator to be scalable, any nonscalar data structures used by the allocator must be stored in the heap itself.

Aligning blocks (alignment requirement). The allocator must align blocks in such a way that they can hold any type of data object.

Not modifying allocated blocks. Allocators can only manipulate or change free blocks. In particular, they are not allowed to modify or move blocks once they are allocated. Thus, techniques such as compaction of allocated blocks are not permitted.

Working within these constraints, the author of an allocator attempts to meet the often conflicting performance goals of maximizing throughput and memory utilization.

Goal 1: Maximizing throughput. Given some sequence of n allocate and free requests

$$R_0, R_1, \dots, R_k, \dots, R_{n-1}$$

we would like to maximize an allocator's *throughput*, which is defined as the number of requests that it completes per unit time. For example, if an allocator completes 500 allocate requests and 500 free requests in 1 second, then its throughput is 1,000 operations per second. In general, we can maximize throughput by minimizing the average time to satisfy allocate and free requests. As we'll see, it is not too difficult to develop allocators with reasonably good performance where the worst-case running time of an allocate request is linear in the number of free blocks and the running time of a free request is constant.

Goal 2: Maximizing memory utilization. Naive programmers often incorrectly assume that virtual memory is an unlimited resource. In fact, the total amount of virtual memory allocated by all of the processes in a system is limited by the amount of swap space on disk. Good programmers know that virtual memory is a finite resource that must be used efficiently. This is especially true for a dynamic memory allocator that might be asked to allocate and free large blocks of memory.

There are a number of ways to characterize how efficiently an allocator uses the heap. In our experience, the most useful metric is *peak utilization*. As before, we are given some sequence of n allocate and free requests

$$R_0, R_1, \dots, R_k, \dots, R_{n-1}$$

If an application requests a block of p bytes, then the resulting allocated block has a *payload* of p bytes. After request R_k has completed, let the *aggregate payload*, denoted P_k , be the sum of the payloads of the currently allocated blocks, and let H_k denote the current (monotonically nondecreasing) size of the heap.

Then the peak utilization over the first $k + 1$ requests, denoted by U_k , is given by

$$U_k = \frac{\max_{i \leq k} P_i}{H_k}$$

The objective of the allocator, then, is to maximize the peak utilization U_{n-1} over the entire sequence. As we will see, there is a tension between maximizing throughput and utilization. In particular, it is easy to write an allocator that maximizes throughput at the expense of heap utilization. One of the interesting challenges in any allocator design is finding an appropriate balance between the two goals.

Aside Relaxing the monotonicity assumption

We could relax the monotonically nondecreasing assumption in our definition of U_k and allow the heap to grow up and down by letting H_k be the high-water mark over the first $k + 1$ requests.

9.9.4 Fragmentation

The primary cause of poor heap utilization is a phenomenon known as *fragmentation*, which occurs when otherwise unused memory is not available to satisfy allocate requests. There are two forms of fragmentation: *internal fragmentation* and *external fragmentation*.

Internal fragmentation occurs when an allocated block is larger than the payload. This might happen for a number of reasons. For example, the implementation of an allocator might impose a minimum size on allocated blocks that is greater than some requested payload. Or, as we saw in Figure 9.34(b), the allocator might increase the block size in order to satisfy alignment constraints.

Internal fragmentation is straightforward to quantify. It is simply the sum of the differences between the sizes of the allocated blocks and their payloads. Thus, at any point in time, the amount of internal fragmentation depends only on the pattern of previous requests and the allocator implementation.

External fragmentation occurs when there is enough aggregate free memory to satisfy an allocate request, but no single free block is large enough to handle the request. For example, if the request in Figure 9.34(e) were for eight words rather than two words, then the request could not be satisfied without requesting additional virtual memory from the kernel, even though there are eight free words remaining in the heap. The problem arises because these eight words are spread over two free blocks.

External fragmentation is much more difficult to quantify than internal fragmentation because it depends not only on the pattern of previous requests and the allocator implementation but also on the pattern of *future* requests. For example, suppose that after k requests all of the free blocks are exactly four words in size. Does this heap suffer from external fragmentation? The answer depends on the pattern of future requests. If all of the future allocate requests are for blocks that are smaller than or equal to four words, then there is no external fragmentation. On the other hand, if one or more requests ask for blocks larger than four words, then the heap does suffer from external fragmentation.

Since external fragmentation is difficult to quantify and impossible to predict, allocators typically employ heuristics that attempt to maintain small numbers of larger free blocks rather than large numbers of smaller free blocks.

9.9.5 Implementation Issues

The simplest imaginable allocator would organize the heap as a large array of bytes and a pointer p that initially points to the first byte of the array. To allocate

size bytes, `malloc` would save the current value of `p` on the stack, increment `p` by size, and return the old value of `p` to the caller. `Free` would simply return to the caller without doing anything.

This naive allocator is an extreme point in the design space. Since each `malloc` and `free` execute only a handful of instructions, throughput would be extremely good. However, since the allocator never reuses any blocks, memory utilization would be extremely bad. A practical allocator that strikes a better balance between throughput and utilization must consider the following issues:

Free block organization. How do we keep track of free blocks?

Placement. How do we choose an appropriate free block in which to place a newly allocated block?

Splitting. After we place a newly allocated block in some free block, what do we do with the remainder of the free block?

Coalescing. What do we do with a block that has just been freed?

The rest of this section looks at these issues in more detail. Since the basic techniques of placement, splitting, and coalescing cut across many different free block organizations, we will introduce them in the context of a simple free block organization known as an implicit free list.

9.9.6 Implicit Free Lists

Any practical allocator needs some data structure that allows it to distinguish block boundaries and to distinguish between allocated and free blocks. Most allocators embed this information in the blocks themselves. One simple approach is shown in Figure 9.35.

In this case, a block consists of a one-word *header*, the payload, and possibly some additional *padding*. The header encodes the block size (including the header and any padding) as well as whether the block is allocated or free. If we impose a double-word alignment constraint, then the block size is always a multiple of 8 and the 3 low-order bits of the block size are always zero. Thus, we need to store only the 29 high-order bits of the block size, freeing the remaining 3 bits to encode other information. In this case, we are using the least significant of these bits

Figure 9.35
Format of a simple heap block.

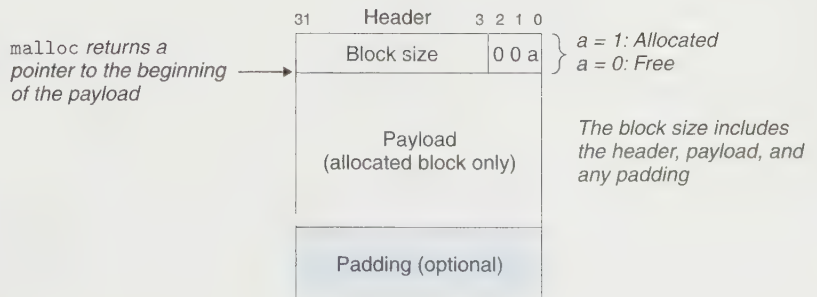




Figure 9.36 Organizing the heap with an implicit free list. Allocated blocks are shaded. Free blocks are unshaded. Headers are labeled with (size (bytes))/allocated bit).

(the *allocated bit*) to indicate whether the block is allocated or free. For example, suppose we have an allocated block with a block size of 24 (0x18) bytes. Then its header would be

```
0x00000018 | 0x1 = 0x00000019
```

Similarly, a free block with a block size of 40 (0x28) bytes would have a header of

```
0x00000028 | 0x0 = 0x00000028
```

The header is followed by the payload that the application requested when it called `malloc`. The payload is followed by a chunk of unused padding that can be any size. There are a number of reasons for the padding. For example, the padding might be part of an allocator’s strategy for combating external fragmentation. Or it might be needed to satisfy the alignment requirement.

Given the block format in Figure 9.35, we can organize the heap as a sequence of contiguous allocated and free blocks, as shown in Figure 9.36.

We call this organization an *implicit free list* because the free blocks are linked implicitly by the size fields in the headers. The allocator can indirectly traverse the entire set of free blocks by traversing *all* of the blocks in the heap. Notice that we need some kind of specially marked end block—in this example, a terminating header with the allocated bit set and a size of zero. (As we will see in Section 9.9.12, setting the allocated bit simplifies the coalescing of free blocks.)

The advantage of an implicit free list is simplicity. A significant disadvantage is that the cost of any operation that requires a search of the free list, such as placing allocated blocks, will be linear in the *total* number of allocated and free blocks in the heap.

It is important to realize that the system’s alignment requirement and the allocator’s choice of block format impose a *minimum block size* on the allocator. No allocated or free block may be smaller than this minimum. For example, if we assume a double-word alignment requirement, then the size of each block must be a multiple of two words (8 bytes). Thus, the block format in Figure 9.35 induces a minimum block size of two words: one word for the header and another to maintain the alignment requirement. Even if the application were to request a single byte, the allocator would still create a two-word block.

Practice Problem 9.6 (solution page 883)

Determine the block sizes and header values that would result from the following sequence of malloc requests. Assumptions: (1) The allocator maintains double-word alignment and uses an implicit free list with the block format from Figure 9.35. (2) Block sizes are rounded up to the nearest multiple of 8 bytes.

Request	Block size (decimal bytes)	Block header (hex)
malloc(1)		
malloc(5)		
malloc(12)		
malloc(13)		

9.9.7 Placing Allocated Blocks

When an application requests a block of k bytes, the allocator searches the free list for a free block that is large enough to hold the requested block. The manner in which the allocator performs this search is determined by the *placement policy*. Some common policies are first fit, next fit, and best fit.

First fit searches the free list from the beginning and chooses the first free block that fits. *Next fit* is similar to first fit, but instead of starting each search at the beginning of the list, it starts each search where the previous search left off. *Best fit* examines every free block and chooses the free block with the smallest size that fits.

An advantage of first fit is that it tends to retain large free blocks at the end of the list. A disadvantage is that it tends to leave “splinters” of small free blocks toward the beginning of the list, which will increase the search time for larger blocks. Next fit was first proposed by Donald Knuth as an alternative to first fit, motivated by the idea that if we found a fit in some free block the last time, there is a good chance that we will find a fit the next time in the remainder of the block. Next fit can run significantly faster than first fit, especially if the front of the list becomes littered with many small splinters. However, some studies suggest that next fit suffers from worse memory utilization than first fit. Studies have found that best fit generally enjoys better memory utilization than either first fit or next fit. However, the disadvantage of using best fit with simple free list organizations such as the implicit free list is that it requires an exhaustive search of the heap. Later, we will look at more sophisticated segregated free list organizations that approximate a best-fit policy without an exhaustive search of the heap.

9.9.8 Splitting Free Blocks

Once the allocator has located a free block that fits, it must make another policy decision about how much of the free block to allocate. One option is to use the entire free block. Although simple and fast, the main disadvantage is that it

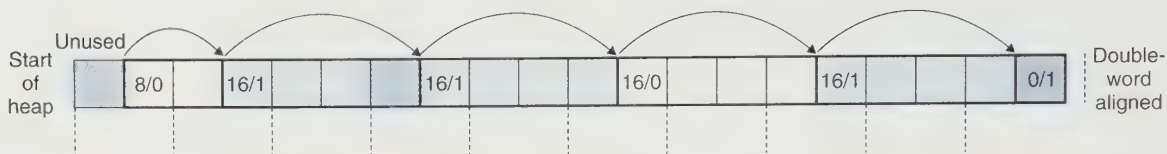


Figure 9.37 Splitting a free block to satisfy a three-word allocation request. Allocated blocks are shaded. Free blocks are unshaded. Headers are labeled with (size (bytes)/allocated bit).

introduces internal fragmentation. If the placement policy tends to produce good fits, then some additional internal fragmentation might be acceptable.

However, if the fit is not good, then the allocator will usually opt to *split* the free block into two parts. The first part becomes the allocated block, and the remainder becomes a new free block. Figure 9.37 shows how the allocator might split the eight-word free block in Figure 9.36 to satisfy an application's request for three words of heap memory.

9.9.9 Getting Additional Heap Memory

What happens if the allocator is unable to find a fit for the requested block? One option is to try to create some larger free blocks by merging (coalescing) free blocks that are physically adjacent in memory (next section). However, if this does not yield a sufficiently large block, or if the free blocks are already maximally coalesced, then the allocator asks the kernel for additional heap memory by calling the `sbrk` function. The allocator transforms the additional memory into one large free block, inserts the block into the free list, and then places the requested block in this new free block.

9.9.10 Coalescing Free Blocks

When the allocator frees an allocated block, there might be other free blocks that are adjacent to the newly freed block. Such adjacent free blocks can cause a phenomenon known as *false fragmentation*, where there is a lot of available free memory chopped up into small, unusable free blocks. For example, Figure 9.38 shows the result of freeing the block that was allocated in Figure 9.37. The result is two adjacent free blocks with payloads of three words each. As a result, a subsequent request for a payload of four words would fail, even though the aggregate size of the two free blocks is large enough to satisfy the request.

To combat false fragmentation, any practical allocator must merge adjacent free blocks in a process known as *coalescing*. This raises an important policy decision about when to perform coalescing. The allocator can opt for *immediate coalescing* by merging any adjacent blocks each time a block is freed. Or it can opt for *deferred coalescing* by waiting to coalesce free blocks at some later time. For example, the allocator might defer coalescing until some allocation request fails, and then scan the entire heap, coalescing all free blocks.

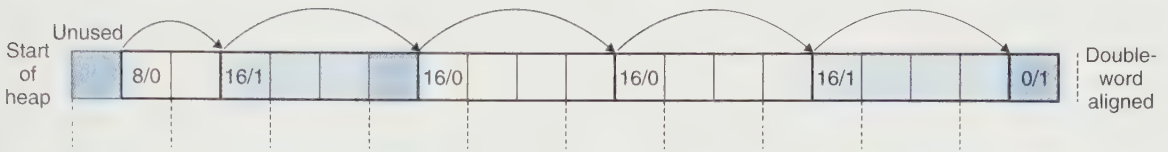


Figure 9.38 An example of false fragmentation. Allocated blocks are shaded. Free blocks are unshaded. Headers are labeled with (size (bytes))/allocated bit).

Immediate coalescing is straightforward and can be performed in constant time, but with some request patterns it can introduce a form of thrashing where a block is repeatedly coalesced and then split soon thereafter. For example, in Figure 9.38, a repeated pattern of allocating and freeing a three-word block would introduce a lot of unnecessary splitting and coalescing. In our discussion of allocators, we will assume immediate coalescing, but you should be aware that fast allocators often opt for some form of deferred coalescing.

9.9.11 Coalescing with Boundary Tags

How does an allocator implement coalescing? Let us refer to the block we want to free as the *current block*. Then coalescing the next free block (in memory) is straightforward and efficient. The header of the current block points to the header of the next block, which can be checked to determine if the next block is free. If so, its size is simply added to the size of the current header and the blocks are coalesced in constant time.

But how would we coalesce the previous block? Given an implicit free list of blocks with headers, the only option would be to search the entire list, remembering the location of the previous block, until we reached the current block. With an implicit free list, this means that each call to `free` would require time linear in the size of the heap. Even with more sophisticated free list organizations, the search time would not be constant.

Knuth developed a clever and general technique, known as *boundary tags*, that allows for constant-time coalescing of the previous block. The idea, which is shown in Figure 9.39, is to add a *footer* (the boundary tag) at the end of each block, where the footer is a replica of the header. If each block includes such a footer, then the allocator can determine the starting location and status of the previous block by inspecting its footer, which is always one word away from the start of the current block.

Consider all the cases that can exist when the allocator frees the current block:

1. The previous and next blocks are both allocated.
2. The previous block is allocated and the next block is free.
3. The previous block is free and the next block is allocated.
4. The previous and next blocks are both free.

Figure 9.39
Format of heap block that
uses a boundary tag.

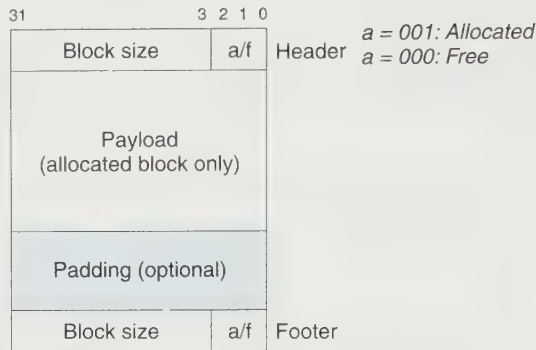


Figure 9.40 shows how we would coalesce each of the four cases.

In case 1, both adjacent blocks are allocated and thus no coalescing is possible. So the status of the current block is simply changed from allocated to free. In case 2, the current block is merged with the next block. The header of the current block and the footer of the next block are updated with the combined sizes of the current and next blocks. In case 3, the previous block is merged with the current block. The header of the previous block and the footer of the current block are updated with the combined sizes of the two blocks. In case 4, all three blocks are merged to form a single free block, with the header of the previous block and the footer of the next block updated with the combined sizes of the three blocks. In each case, the coalescing is performed in constant time.

The idea of boundary tags is a simple and elegant one that generalizes to many different types of allocators and free list organizations. However, there is a potential disadvantage. Requiring each block to contain both a header and a footer can introduce significant memory overhead if an application manipulates many small blocks. For example, if a graph application dynamically creates and destroys graph nodes by making repeated calls to `malloc` and `free`, and each graph node requires only a couple of words of memory, then the header and the footer will consume half of each allocated block.

Fortunately, there is a clever optimization of boundary tags that eliminates the need for a footer in allocated blocks. Recall that when we attempt to coalesce the current block with the previous and next blocks in memory, the size field in the footer of the previous block is only needed if the previous block is *free*. If we were to store the allocated/free bit of the previous block in one of the excess low-order bits of the current block, then allocated blocks would not need footers, and we could use that extra space for payload. Note, however, that free blocks would still need footers.

Practice Problem 9.7 (solution page 883)

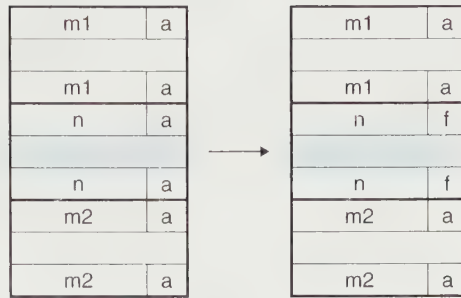
Determine the minimum block size for each of the following combinations of alignment requirements and block formats. Assumptions: Implicit free list, zero-size payloads are not allowed, and headers and footers are stored in 4-byte words.

Figure 9.40

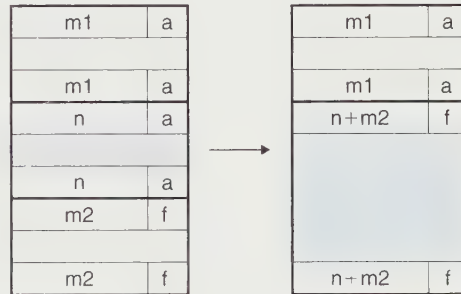
Coalescing with

boundary tags. Case 1:
prev and next allocated.

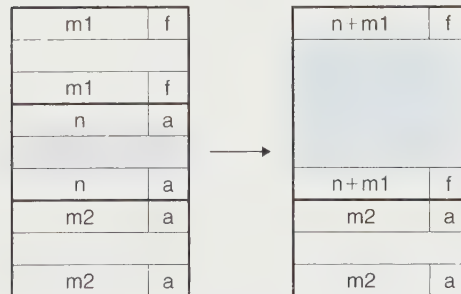
Case 2: prev allocated, next
free. Case 3: prev free, next
allocated. Case 4: next and
prev free.



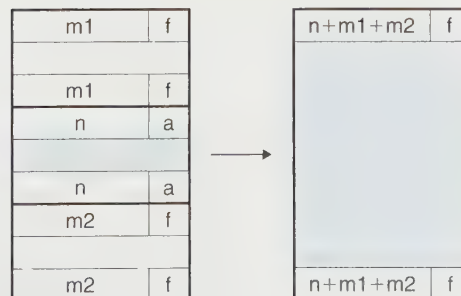
Case 1



Case 2



Case 3



Case 4

Alignment	Allocated block	Free block	Minimum block size (bytes)
Single word	Header and footer	Header and footer	_____
Single word	Header, but no footer	Header and footer	_____
Double word	Header and footer	Header and footer	_____
Double word	Header, but no footer	Header and footer	_____

9.9.12 Putting It Together: Implementing a Simple Allocator

Building an allocator is a challenging task. The design space is large, with numerous alternatives for block format and free list format, as well as placement, splitting, and coalescing policies. Another challenge is that you are often forced to program outside the safe, familiar confines of the type system, relying on the error-prone pointer casting and pointer arithmetic that is typical of low-level systems programming.

While allocators do not require enormous amounts of code, they are subtle and unforgiving. Students familiar with higher-level languages such as C++ or Java often hit a conceptual wall when they first encounter this style of programming. To help you clear this hurdle, we will work through the implementation of a simple allocator based on an implicit free list with immediate boundary-tag coalescing. The maximum block size is $2^{32} = 4$ GB. The code is 64-bit clean, running without modification in 32-bit (`gcc -m32`) or 64-bit (`gcc -m64`) processes.

General Allocator Design

Our allocator uses a model of the memory system provided by the `memlib.c` package shown in Figure 9.41. The purpose of the model is to allow us to run our allocator without interfering with the existing system-level `malloc` package.

The `mem_init` function models the virtual memory available to the heap as a large double-word aligned array of bytes. The bytes between `mem_heap` and `mem_brk` represent allocated virtual memory. The bytes following `mem_brk` represent unallocated virtual memory. The allocator requests additional heap memory by calling the `mem_sbrk` function, which has the same interface as the system's `sbrk` function, as well as the same semantics, except that it rejects requests to shrink the heap.

The allocator itself is contained in a source file (`mm.c`) that users can compile and link into their applications. The allocator exports three functions to application programs:

```

1  extern int mm_init(void);
2  extern void *mm_malloc (size_t size);
3  extern void mm_free (void *ptr);
```

The `mm_init` function initializes the allocator, returning 0 if successful and -1 otherwise. The `mm_malloc` and `mm_free` functions have the same interfaces and semantics as their system counterparts. The allocator uses the block format

```

1  /* Private global variables */
2  static char *mem_heap;      /* Points to first byte of heap */
3  static char *mem_brk;      /* Points to last byte of heap plus 1 */
4  static char *mem_max_addr; /* Max legal heap addr plus 1*/
5
6  /*
7   * mem_init - Initialize the memory system model
8   */
9  void mem_init(void)
10 {
11     mem_heap = (char *)Malloc(MAX_HEAP);
12     mem_brk = (char *)mem_heap;
13     mem_max_addr = (char *) (mem_heap + MAX_HEAP);
14 }
15
16 /*
17 * mem_sbrk - Simple model of the sbrk function. Extends the heap
18 *   by incr bytes and returns the start address of the new area. In
19 *   this model, the heap cannot be shrunk.
20 */
21 void *mem_sbrk(int incr)
22 {
23     char *old_brk = mem_brk;
24
25     if ( (incr < 0) || ((mem_brk + incr) > mem_max_addr)) {
26         errno = ENOMEM;
27         fprintf(stderr, "ERROR: mem_sbrk failed. Ran out of memory...\n");
28         return (void *)-1;
29     }
30     mem_brk += incr;
31     return (void *)old_brk;
32 }

```

Figure 9.41 memlib.c: Memory system model.

shown in Figure 9.39. The minimum block size is 16 bytes. The free list is organized as an implicit free list, with the invariant form shown in Figure 9.42.

The first word is an unused padding word aligned to a double-word boundary. The padding is followed by a special *prologue block*, which is an 8-byte allocated block consisting of only a header and a footer. The prologue block is created during initialization and is never freed. Following the prologue block are zero or more regular blocks that are created by calls to `malloc` or `free`. The heap always ends with a special *epilogue block*, which is a zero-size allocated block

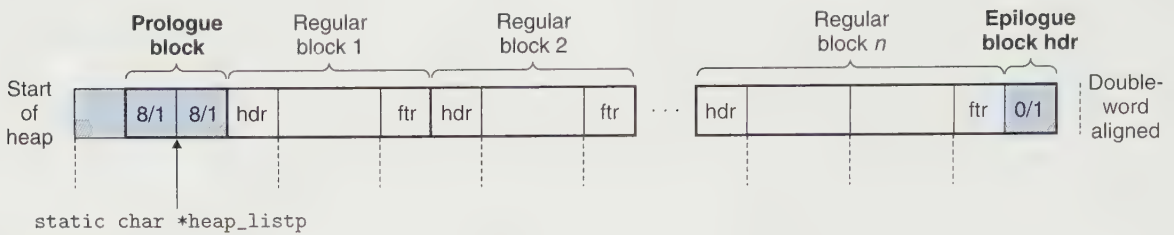


Figure 9.42 Invariant form of the implicit free list.

that consists of only a header. The prologue and epilogue blocks are tricks that eliminate the edge conditions during coalescing. The allocator uses a single private (static) global variable (`heap_listp`) that always points to the prologue block. (As a minor optimization, we could make it point to the next block instead of the prologue block.)

Basic Constants and Macros for Manipulating the Free List

Figure 9.43 shows some basic constants and macros that we will use throughout the allocator code. Lines 2–4 define some basic size constants: the sizes of words (`WSIZE`) and double words (`DSIZE`), and the size of the initial free block and the default size for expanding the heap (`CHUNKSIZE`).

Manipulating the headers and footers in the free list can be troublesome because it demands extensive use of casting and pointer arithmetic. Thus, we find it helpful to define a small set of macros for accessing and traversing the free list (lines 9–25). The `PACK` macro (line 9) combines a size and an allocate bit and returns a value that can be stored in a header or footer.

The `GET` macro (line 12) reads and returns the word referenced by argument `p`. The casting here is crucial. The argument `p` is typically a `(void *)` pointer, which cannot be dereferenced directly. Similarly, the `PUT` macro (line 13) stores `val` in the word pointed at by argument `p`.

The `GET_SIZE` and `GET_ALLOC` macros (lines 16–17) return the size and allocated bit, respectively, from a header or footer at address `p`. The remaining macros operate on *block pointers* (denoted `bp`) that point to the first payload byte. Given a block pointer `bp`, the `HDRP` and `FTRP` macros (lines 20–21) return pointers to the block header and footer, respectively. The `NEXT_BLKP` and `PREV_BLKP` macros (lines 24–25) return the block pointers of the next and previous blocks, respectively.

The macros can be composed in various ways to manipulate the free list. For example, given a pointer `bp` to the current block, we could use the following line of code to determine the size of the next block in memory:

```
size_t size = GET_SIZE(HDRP(NEXT_BLKP(bp)));
```

code/vm/malloc/mm.c

```

1  /* Basic constants and macros */
2  #define WSIZE      4      /* Word and header/footer size (bytes) */
3  #define DSIZE      8      /* Double word size (bytes) */
4  #define CHUNKSIZE (1<<12) /* Extend heap by this amount (bytes) */
5
6  #define MAX(x, y) ((x) > (y)? (x) : (y))
7
8  /* Pack a size and allocated bit into a word */
9  #define PACK(size, alloc) ((size) | (alloc))
10
11 /* Read and write a word at address p */
12 #define GET(p)      (*(unsigned int *)(p))
13 #define PUT(p, val) (*(unsigned int *)(p) = (val))
14
15 /* Read the size and allocated fields from address p */
16 #define GET_SIZE(p) (GET(p) & ~0x7)
17 #define GET_ALLOC(p) (GET(p) & 0x1)
18
19 /* Given block ptr bp, compute address of its header and footer */
20 #define HDRP(bp)     ((char *) (bp) - WSIZE)
21 #define FTRP(bp)     ((char *) (bp) + GET_SIZE(HDRP(bp)) - DSIZE)
22
23 /* Given block ptr bp, compute address of next and previous blocks */
24 #define NEXT_BLKP(bp) ((char *) (bp) + GET_SIZE(((char *) (bp) - WSIZE)))
25 #define PREV_BLKP(bp) ((char *) (bp) - GET_SIZE(((char *) (bp) - DSIZE)))

```

*code/vm/malloc/mm.c***Figure 9.43** Basic constants and macros for manipulating the free list.

Creating the Initial Free List

Before calling `mm_malloc` or `mm_free`, the application must initialize the heap by calling the `mm_init` function (Figure 9.44).

The `mm_init` function gets four words from the memory system and initializes them to create the empty free list (lines 4–10). It then calls the `extend_heap` function (Figure 9.45), which extends the heap by `CHUNKSIZE` bytes and creates the initial free block. At this point, the allocator is initialized and ready to accept allocate and free requests from the application.

The `extend_heap` function is invoked in two different circumstances: (1) when the heap is initialized and (2) when `mm_malloc` is unable to find a suitable fit. To maintain alignment, `extend_heap` rounds up the requested size to the nearest

code/vm/malloc/mm.c

```

1  int mm_init(void)
2  {
3      /* Create the initial empty heap */
4      if ((heap_listp = mem_sbrk(4*WSIZE)) == (void *)-1)
5          return -1;
6      PUT(heap_listp, 0);                          /* Alignment padding */
7      PUT(heap_listp + (1*WSIZE), PACK(DSIZE, 1)); /* Prologue header */
8      PUT(heap_listp + (2*WSIZE), PACK(DSIZE, 1)); /* Prologue footer */
9      PUT(heap_listp + (3*WSIZE), PACK(0, 1));      /* Epilogue header */
10     heap_listp += (2*WSIZE);
11
12     /* Extend the empty heap with a free block of CHUNKSIZE bytes */
13     if (extend_heap(CHUNKSIZE/WSIZE) == NULL)
14         return -1;
15     return 0;
16 }

```

*code/vm/malloc/mm.c***Figure 9.44** `mm_init` creates a heap with an initial free block.*code/vm/malloc/mm.c*

```

1  static void *extend_heap(size_t words)
2  {
3      char *bp;
4      size_t size;
5
6      /* Allocate an even number of words to maintain alignment */
7      size = (words % 2) ? (words+1) * WSIZE : words * WSIZE;
8      if ((long)(bp = mem_sbrk(size)) == -1)
9          return NULL;
10
11     /* Initialize free block header/footer and the epilogue header */
12     PUT(HDRP(bp), PACK(size, 0));          /* Free block header */
13     PUT(FTRP(bp), PACK(size, 0));          /* Free block footer */
14     PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1)); /* New epilogue header */
15
16     /* Coalesce if the previous block was free */
17     return coalesce(bp);
18 }

```

*code/vm/malloc/mm.c***Figure 9.45** `extend_heap` extends the heap with a new free block.

multiple of 2 words (8 bytes) and then requests the additional heap space from the memory system (lines 7–9).

The remainder of the `extend_heap` function (lines 12–17) is somewhat subtle. The heap begins on a double-word aligned boundary, and every call to `extend_heap` returns a block whose size is an integral number of double words. Thus, every call to `mem_sbrk` returns a double-word aligned chunk of memory immediately following the header of the epilogue block. This header becomes the header of the new free block (line 12), and the last word of the chunk becomes the new epilogue block header (line 14). Finally, in the likely case that the previous heap was terminated by a free block, we call the `coalesce` function to merge the two free blocks and return the block pointer of the merged blocks (line 17).

Freeing and Coalescing Blocks

An application frees a previously allocated block by calling the `mm_free` function (Figure 9.46), which frees the requested block (`bp`) and then merges adjacent free blocks using the boundary-tags coalescing technique described in Section 9.9.11.

The code in the `coalesce` helper function is a straightforward implementation of the four cases outlined in Figure 9.40. There is one somewhat subtle aspect. The free list format we have chosen—with its prologue and epilogue blocks that are always marked as allocated—allows us to ignore the potentially troublesome edge conditions where the requested block `bp` is at the beginning or end of the heap. Without these special blocks, the code would be messier, more error prone, and slower because we would have to check for these rare edge conditions on each and every free request.

Allocating Blocks

An application requests a block of size bytes of memory by calling the `mm_malloc` function (Figure 9.47). After checking for spurious requests, the allocator must adjust the requested block size to allow room for the header and the footer, and to satisfy the double-word alignment requirement. Lines 12–13 enforce the minimum block size of 16 bytes: 8 bytes to satisfy the alignment requirement and 8 more bytes for the overhead of the header and footer. For requests over 8 bytes (line 15), the general rule is to add in the overhead bytes and then round up to the nearest multiple of 8.

Once the allocator has adjusted the requested size, it searches the free list for a suitable free block (line 18). If there is a fit, then the allocator places the requested block and optionally splits the excess (line 19) and then returns the address of the newly allocated block.

If the allocator cannot find a fit, it extends the heap with a new free block (lines 24–26), places the requested block in the new free block, optionally splitting the block (line 27), and then returns a pointer to the newly allocated block.

```

1  void mm_free(void *bp)
2  {
3      size_t size = GET_SIZE(HDRP(bp));
4
5      PUT(HDRP(bp), PACK(size, 0));
6      PUT(FTRP(bp), PACK(size, 0));
7      coalesce(bp);
8  }
9
10 static void *coalesce(void *bp)
11 {
12     size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKBP(bp)));
13     size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKBP(bp)));
14     size_t size = GET_SIZE(HDRP(bp));
15
16     if (prev_alloc && next_alloc) {          /* Case 1 */
17         return bp;
18     }
19
20     else if (prev_alloc && !next_alloc) {     /* Case 2 */
21         size += GET_SIZE(HDRP(NEXT_BLKBP(bp)));
22         PUT(HDRP(bp), PACK(size, 0));
23         PUT(FTRP(bp), PACK(size, 0));
24     }
25
26     else if (!prev_alloc && next_alloc) {     /* Case 3 */
27         size += GET_SIZE(HDRP(PREV_BLKBP(bp)));
28         PUT(FTRP(bp), PACK(size, 0));
29         PUT(HDRP(PREV_BLKBP(bp)), PACK(size, 0));
30         bp = PREV_BLKBP(bp);
31     }
32
33     else {                                   /* Case 4 */
34         size += GET_SIZE(HDRP(PREV_BLKBP(bp))) +
35             GET_SIZE(FTRP(NEXT_BLKBP(bp)));
36         PUT(HDRP(PREV_BLKBP(bp)), PACK(size, 0));
37         PUT(FTRP(NEXT_BLKBP(bp)), PACK(size, 0));
38         bp = PREV_BLKBP(bp);
39     }
40     return bp;
41 }

```

Figure 9.46 `mm_free` frees a block and uses boundary-tag coalescing to merge it with any adjacent free blocks in constant time.

code/vm/malloc/mm.c

```

1 void *mm_malloc(size_t size)
2 {
3     size_t asize;      /* Adjusted block size */
4     size_t extendsize; /* Amount to extend heap if no fit */
5     char *bp;
6
7     /* Ignore spurious requests */
8     if (size == 0)
9         return NULL;
10
11    /* Adjust block size to include overhead and alignment reqs. */
12    if (size <= DSIZE)
13        asize = 2*DSIZE;
14    else
15        asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);
16
17    /* Search the free list for a fit */
18    if ((bp = find_fit(asize)) != NULL) {
19        place(bp, asize);
20        return bp;
21    }
22
23    /* No fit found. Get more memory and place the block */
24    extendsize = MAX(asize, CHUNKSIZE);
25    if ((bp = extend_heap(extendsize/WSIZE)) == NULL)
26        return NULL;
27    place(bp, asize);
28    return bp;
29 }

```

*code/vm/malloc/mm.c***Figure 9.47** `mm_malloc` allocates a block from the free list.**Practice Problem 9.8** (solution page 884)

Implement a `find_fit` function for the simple allocator described in Section 9.9.12.

```
static void *find_fit(size_t asize)
```

Your solution should perform a first-fit search of the implicit free list.

Practice Problem 9.9 (solution page 884)

Implement a `place` function for the example allocator.

```

static void place(void *bp, size_t asize)

```

Your solution should place the requested block at the beginning of the free block, splitting only if the size of the remainder would equal or exceed the minimum block size.

9.9.13 Explicit Free Lists

The implicit free list provides us with a simple way to introduce some basic allocator concepts. However, because block allocation time is linear in the total number of heap blocks, the implicit free list is not appropriate for a general-purpose allocator (although it might be fine for a special-purpose allocator where the number of heap blocks is known beforehand to be small).

A better approach is to organize the free blocks into some form of explicit data structure. Since by definition the body of a free block is not needed by the program, the pointers that implement the data structure can be stored within the bodies of the free blocks. For example, the heap can be organized as a doubly linked free list by including a *pred* (predecessor) and *succ* (successor) pointer in each free block, as shown in Figure 9.48.

Using a doubly linked list instead of an implicit free list reduces the first-fit allocation time from linear in the total number of blocks to linear in the number of *free* blocks. However, the time to free a block can be either linear or constant, depending on the policy we choose for ordering the blocks in the free list.

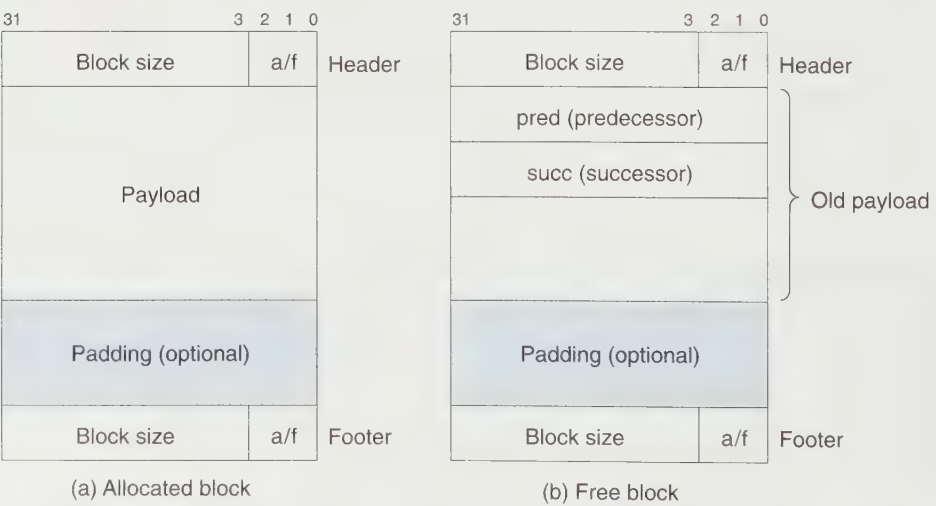


Figure 9.48 Format of heap blocks that use doubly linked free lists.

One approach is to maintain the list in *last-in first-out* (LIFO) order by inserting newly freed blocks at the beginning of the list. With a LIFO ordering and a first-fit placement policy, the allocator inspects the most recently used blocks first. In this case, freeing a block can be performed in constant time. If boundary tags are used, then coalescing can also be performed in constant time.

Another approach is to maintain the list in *address order*, where the address of each block in the list is less than the address of its successor. In this case, freeing a block requires a linear-time search to locate the appropriate predecessor. The trade-off is that address-ordered first fit enjoys better memory utilization than LIFO-ordered first fit, approaching the utilization of best fit.

A disadvantage of explicit lists in general is that free blocks must be large enough to contain all of the necessary pointers, as well as the header and possibly a footer. This results in a larger minimum block size and increases the potential for internal fragmentation.

9.9.14 Segregated Free Lists

As we have seen, an allocator that uses a single linked list of free blocks requires time linear in the number of free blocks to allocate a block. A popular approach for reducing the allocation time, known generally as *segregated storage*, is to maintain multiple free lists, where each list holds blocks that are roughly the same size. The general idea is to partition the set of all possible block sizes into equivalence classes called *size classes*. There are many ways to define the size classes. For example, we might partition the block sizes by powers of 2:

$$\{1\}, \{2\}, \{3, 4\}, \{5-8\}, \dots, \{1,025-2,048\}, \{2,049-4,096\}, \{4,097-\infty\}$$

Or we might assign small blocks to their own size classes and partition large blocks by powers of 2:

$$\{1\}, \{2\}, \{3\}, \dots, \{1,023\}, \{1,024\}, \{1,025-2,048\}, \{2,049-4,096\}, \{4,097-\infty\}$$

The allocator maintains an array of free lists, with one free list per size class, ordered by increasing size. When the allocator needs a block of size n , it searches the appropriate free list. If it cannot find a block that fits, it searches the next list, and so on.

The dynamic storage allocation literature describes dozens of variants of segregated storage that differ in how they define size classes, when they perform coalescing, when they request additional heap memory from the operating system, whether they allow splitting, and so forth. To give you a sense of what is possible, we will describe two of the basic approaches: *simple segregated storage* and *segregated fits*.

Simple Segregated Storage

With simple segregated storage, the free list for each size class contains same-size blocks, each the size of the largest element of the size class. For example, if some size class is defined as {17–32}, then the free list for that class consists entirely of blocks of size 32.

To allocate a block of some given size, we check the appropriate free list. If the list is not empty, we simply allocate the first block in its entirety. Free blocks are never split to satisfy allocation requests. If the list is empty, the allocator requests a fixed-size chunk of additional memory from the operating system (typically a multiple of the page size), divides the chunk into equal-size blocks, and links the blocks together to form the new free list. To free a block, the allocator simply inserts the block at the front of the appropriate free list.

There are a number of advantages to this simple scheme. Allocating and freeing blocks are both fast constant-time operations. Further, the combination of the same-size blocks in each chunk, no splitting, and no coalescing means that there is very little per-block memory overhead. Since each chunk has only same-size blocks, the size of an allocated block can be inferred from its address. Since there is no coalescing, allocated blocks do not need an allocated/free flag in the header. Thus, allocated blocks require no headers, and since there is no coalescing, they do not require any footers either. Since allocate and free operations insert and delete blocks at the beginning of the free list, the list need only be singly linked instead of doubly linked. The bottom line is that the only required field in any block is a one-word succ pointer in each free block, and thus the minimum block size is only one word.

A significant disadvantage is that simple segregated storage is susceptible to internal and external fragmentation. Internal fragmentation is possible because free blocks are never split. Worse, certain reference patterns can cause extreme external fragmentation because free blocks are never coalesced (Practice Problem 9.10).

Practice Problem 9.10 (solution page 885)

Describe a reference pattern that results in severe external fragmentation in an allocator based on simple segregated storage.

Segregated Fits

With this approach, the allocator maintains an array of free lists. Each free list is associated with a size class and is organized as some kind of explicit or implicit list. Each list contains potentially different-size blocks whose sizes are members of the size class. There are many variants of segregated fits allocators. Here we describe a simple version.

To allocate a block, we determine the size class of the request and do a first-fit search of the appropriate free list for a block that fits. If we find one, then we (optionally) split it and insert the fragment in the appropriate free list. If we cannot find a block that fits, then we search the free list for the next larger size class. We

repeat until we find a block that fits. If none of the free lists yields a block that fits, then we request additional heap memory from the operating system, allocate the block out of this new heap memory, and place the remainder in the appropriate size class. To free a block, we coalesce and place the result on the appropriate free list.

The segregated fits approach is a popular choice with production-quality allocators such as the GNU `malloc` package provided in the C standard library because it is both fast and memory efficient. Search times are reduced because searches are limited to particular parts of the heap instead of the entire heap. Memory utilization can improve because of the interesting fact that a simple first-fit search of a segregated free list approximates a best-fit search of the entire heap.

Buddy Systems

A *buddy system* is a special case of segregated fits where each size class is a power of 2. The basic idea is that, given a heap of 2^m words, we maintain a separate free list for each block size 2^k , where $0 \leq k \leq m$. Requested block sizes are rounded up to the nearest power of 2. Originally, there is one free block of size 2^m words.

To allocate a block of size 2^k , we find the first available block of size 2^j , such that $k \leq j \leq m$. If $j = k$, then we are done. Otherwise, we recursively split the block in half until $j = k$. As we perform this splitting, each remaining half (known as a *buddy*) is placed on the appropriate free list. To free a block of size 2^k , we continue coalescing with the free buddies. When we encounter an allocated buddy, we stop the coalescing.

A key fact about buddy systems is that, given the address and size of a block, it is easy to compute the address of its buddy. For example, a block of size 32 bytes with address

`xxx . . . x00000`

has its buddy at address

`xxx . . . x10000`

In other words, the addresses of a block and its buddy differ in exactly one bit position.

The major advantage of a buddy system allocator is its fast searching and coalescing. The major disadvantage is that the power-of-2 requirement on the block size can cause significant internal fragmentation. For this reason, buddy system allocators are not appropriate for general-purpose workloads. However, for certain application-specific workloads, where the block sizes are known in advance to be powers of 2, buddy system allocators have a certain appeal.

9.10 Garbage Collection

With an explicit allocator such as the C `malloc` package, an application allocates and frees heap blocks by making calls to `malloc` and `free`. It is the application's responsibility to free any allocated blocks that it no longer needs.

Failing to free allocated blocks is a common programming error. For example, consider the following C function that allocates a block of temporary storage as part of its processing:

```

1 void garbage()
2 {
3     int *p = (int *)Malloc(15213);
4
5     return; /* Array p is garbage at this point */
6 }
```

Since *p* is no longer needed by the program, it should have been freed before *garbage* returned. Unfortunately, the programmer has forgotten to free the block. It remains allocated for the lifetime of the program, needlessly occupying heap space that could be used to satisfy subsequent allocation requests.

A *garbage collector* is a dynamic storage allocator that automatically frees allocated blocks that are no longer needed by the program. Such blocks are known as *garbage* (hence the term “garbage collector”). The process of automatically reclaiming heap storage is known as *garbage collection*. In a system that supports garbage collection, applications explicitly allocate heap blocks but never explicitly free them. In the context of a C program, the application calls `malloc` but never calls `free`. Instead, the garbage collector periodically identifies the garbage blocks and makes the appropriate calls to `free` to place those blocks back on the free list.

Garbage collection dates back to Lisp systems developed by John McCarthy at MIT in the early 1960s. It is an important part of modern language systems such as Java, ML, Perl, and Mathematica, and it remains an active and important area of research. The literature describes an amazing number of approaches for garbage collection. We will limit our discussion to McCarthy’s original *Mark&Sweep* algorithm, which is interesting because it can be built on top of an existing `malloc` package to provide garbage collection for C and C++ programs.

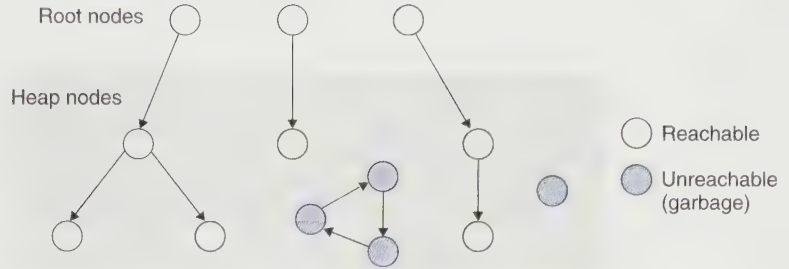
9.10.1 Garbage Collector Basics

A garbage collector views memory as a directed *reachability graph* of the form shown in Figure 9.49. The nodes of the graph are partitioned into a set of *root nodes* and a set of *heap nodes*. Each heap node corresponds to an allocated block in the heap. A directed edge $p \rightarrow q$ means that some location in block *p* points to some location in block *q*. Root nodes correspond to locations not in the heap that contain pointers into the heap. These locations can be registers, variables on the stack, or global variables in the read/write data area of virtual memory.

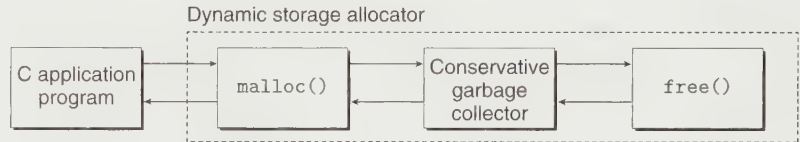
We say that a node *p* is *reachable* if there exists a directed path from any root node to *p*. At any point in time, the unreachable nodes correspond to garbage that can never be used again by the application. The role of a garbage collector is to maintain some representation of the reachability graph and periodically reclaim the unreachable nodes by freeing them and returning them to the free list.

Figure 9.49

A garbage collector's view of memory as a directed graph.

**Figure 9.50**

Integrating a conservative garbage collector and a C malloc package.



Garbage collectors for languages like ML and Java, which exert tight control over how applications create and use pointers, can maintain an exact representation of the reachability graph and thus can reclaim all garbage. However, collectors for languages like C and C++ cannot in general maintain exact representations of the reachability graph. Such collectors are known as *conservative garbage collectors*. They are conservative in the sense that each reachable block is correctly identified as reachable, while some unreachable nodes might be incorrectly identified as reachable.

Collectors can provide their service on demand, or they can run as separate threads in parallel with the application, continuously updating the reachability graph and reclaiming garbage. For example, consider how we might incorporate a conservative collector for C programs into an existing malloc package, as shown in Figure 9.50.

The application calls malloc in the usual manner whenever it needs heap space. If malloc is unable to find a free block that fits, then it calls the garbage collector in hopes of reclaiming some garbage to the free list. The collector identifies the garbage blocks and returns them to the heap by calling the free function. The key idea is that the collector calls free instead of the application. When the call to the collector returns, malloc tries again to find a free block that fits. If that fails, then it can ask the operating system for additional memory. Eventually, malloc returns a pointer to the requested block (if successful) or the NULL pointer (if unsuccessful).

9.10.2 Mark&Sweep Garbage Collectors

A Mark&Sweep garbage collector consists of a *mark phase*, which marks all reachable and allocated descendants of the root nodes, followed by a *sweep phase*, which frees each unmarked allocated block. Typically, one of the spare low-order bits in the block header is used to indicate whether a block is marked or not.

(a) mark function

```

void mark(ptr p) {
    if ((b = isPtr(p)) == NULL)
        return;
    if (blockMarked(b))
        return;
    markBlock(b);
    len = length(b);
    for (i=0; i < len; i++)
        mark(b[i]);
    return;
}

```

(b) sweep function

```

void sweep(ptr b, ptr end) {
    while (b < end) {
        if (blockMarked(b))
            unmarkBlock(b);
        else if (blockAllocated(b))
            free(b);
        b = nextBlock(b);
    }
    return;
}

```

Figure 9.51 Pseudocode for the mark and sweep functions.

Our description of Mark&Sweep will assume the following functions, where `ptr` is defined as `typedef void *ptr`:

`ptr isPtr(ptr p)`. If `p` points to some word in an allocated block, it returns a pointer `b` to the beginning of that block. Returns `NULL` otherwise.

`int blockMarked(ptr b)`. Returns true if block `b` is already marked.

`int blockAllocated(ptr b)`. Returns true if block `b` is allocated.

`void markBlock(ptr b)`. Marks block `b`.

`int length(ptr b)`. Returns the length in words (excluding the header) of block `b`.

`void unmarkBlock(ptr b)`. Changes the status of block `b` from marked to unmarked.

`ptr nextBlock(ptr b)`. Returns the successor of block `b` in the heap.

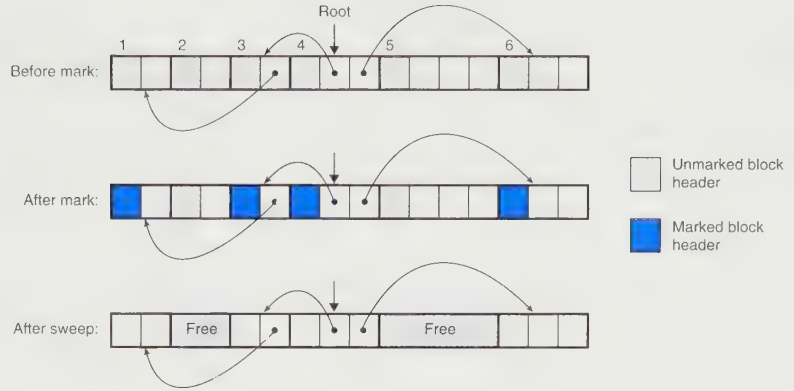
The mark phase calls the mark function shown in Figure 9.51(a) once for each root node. The mark function returns immediately if `p` does not point to an allocated and unmarked heap block. Otherwise, it marks the block and calls itself recursively on each word in block. Each call to the mark function marks any unmarked and reachable descendants of some root node. At the end of the mark phase, any allocated block that is not marked is guaranteed to be unreachable and, hence, garbage that can be reclaimed in the sweep phase.

The sweep phase is a single call to the sweep function shown in Figure 9.51(b). The sweep function iterates over each block in the heap, freeing any unmarked allocated blocks (i.e., garbage) that it encounters.

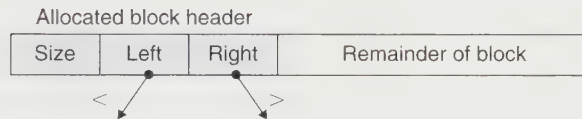
Figure 9.52 shows a graphical interpretation of Mark&Sweep for a small heap. Block boundaries are indicated by heavy lines. Each square corresponds to a word of memory. Each block has a one-word header, which is either marked or unmarked.

Figure 9.52**Mark&Sweep example.**

Note that the arrows in this example denote memory references, not free list pointers.

**Figure 9.53**

Left and right pointers in a balanced tree of allocated blocks.



Initially, the heap in Figure 9.52 consists of six allocated blocks, each of which is unmarked. Block 3 contains a pointer to block 1. Block 4 contains pointers to blocks 3 and 6. The root points to block 4. After the mark phase, blocks 1, 3, 4, and 6 are marked because they are reachable from the root. Blocks 2 and 5 are unmarked because they are unreachable. After the sweep phase, the two unreachable blocks are reclaimed to the free list.

9.10.3 Conservative Mark&Sweep for C Programs

Mark&Sweep is an appropriate approach for garbage collecting C programs because it works in place without moving any blocks. However, the C language poses some interesting challenges for the implementation of the `isPtr` function.

First, C does not tag memory locations with any type information. Thus, there is no obvious way for `isPtr` to determine if its input parameter `p` is a pointer or not. Second, even if we were to know that `p` was a pointer, there would be no obvious way for `isPtr` to determine whether `p` points to some location in the payload of an allocated block.

One solution to the latter problem is to maintain the set of allocated blocks as a balanced binary tree that maintains the invariant that all blocks in the left subtree are located at smaller addresses and all blocks in the right subtree are located in larger addresses. As shown in Figure 9.53, this requires two additional fields (`left` and `right`) in the header of each allocated block. Each field points to the header of some allocated block. The `isPtr(ptr p)` function uses the tree to perform a binary search of the allocated blocks. At each step, it relies on the size field in the block header to determine if `p` falls within the extent of the block.

The balanced tree approach is correct in the sense that it is guaranteed to mark all of the nodes that are reachable from the roots. This is a necessary guarantee, as application users would certainly not appreciate having their allocated blocks prematurely returned to the free list. However, it is conservative in the sense that it may incorrectly mark blocks that are actually unreachable, and thus it may fail to free some garbage. While this does not affect the correctness of application programs, it can result in unnecessary external fragmentation.

The fundamental reason that Mark&Sweep collectors for C programs must be conservative is that the C language does not tag memory locations with type information. Thus, scalars like `ints` or `floats` can masquerade as pointers. For example, suppose that some reachable allocated block contains an `int` in its payload whose value happens to correspond to an address in the payload of some other allocated block *b*. There is no way for the collector to infer that the data is really an `int` and not a pointer. Therefore, the allocator must conservatively mark block *b* as reachable, when in fact it might not be.

9.11 Common Memory-Related Bugs in C Programs

Managing and using virtual memory can be a difficult and error-prone task for C programmers. Memory-related bugs are among the most frightening because they often manifest themselves at a distance, in both time and space, from the source of the bug. Write the wrong data to the wrong location, and your program can run for hours before it finally fails in some distant part of the program. We conclude our discussion of virtual memory with a look at some of the common memory-related bugs.

9.11.1 Dereferencing Bad Pointers

As we learned in Section 9.7.2, there are large holes in the virtual address space of a process that are not mapped to any meaningful data. If we attempt to dereference a pointer into one of these holes, the operating system will terminate our program with a segmentation exception. Also, some areas of virtual memory are read-only. Attempting to write to one of these areas terminates the program with a protection exception.

A common example of dereferencing a bad pointer is the classic `scanf` bug. Suppose we want to use `scanf` to read an integer from `stdin` into a variable. The correct way to do this is to pass `scanf` a format string and the *address* of the variable:

```
scanf("%d", &val)
```

However, it is easy for new C programmers (and experienced ones too!) to pass the *contents* of `val` instead of its address:

```
scanf("%d", val)
```

In this case, `scanf` will interpret the contents of `val` as an address and attempt to write a word to that location. In the best case, the program terminates immediately with an exception. In the worst case, the contents of `val` correspond to some valid read/write area of virtual memory, and we overwrite memory, usually with disastrous and baffling consequences much later.

9.11.2 Reading Uninitialized Memory

While bss memory locations (such as uninitialized global C variables) are always initialized to zeros by the loader, this is not true for heap memory. A common error is to assume that heap memory is initialized to zero:

```

1  /* Return y = Ax */
2  int *matvec(int **A, int *x, int n)
3  {
4      int i, j;
5
6      int *y = (int *)Malloc(n * sizeof(int));
7
8      for (i = 0; i < n; i++)
9          for (j = 0; j < n; j++)
10             y[i] += A[i][j] * x[j];
11     return y;
12 }
```

In this example, the programmer has incorrectly assumed that vector `y` has been initialized to zero. A correct implementation would explicitly zero `y[i]` or use `calloc`.

9.11.3 Allowing Stack Buffer Overflows

As we saw in Section 3.10.3, a program has a *buffer overflow bug* if it writes to a target buffer on the stack without examining the size of the input string. For example, the following function has a buffer overflow bug because the `gets` function copies an arbitrary-length string to the buffer. To fix this, we would need to use the `fgets` function, which limits the size of the input string.

```

1  void bufoverflow()
2  {
3      char buf[64];
4
5      gets(buf); /* Here is the stack buffer overflow bug */
6      return;
7  }
```

9.11.4 Assuming That Pointers and the Objects They Point to Are the Same Size

One common mistake is to assume that pointers to objects are the same size as the objects they point to:

```

1  /* Create an nxm array */
2  int **makeArray1(int n, int m)
3  {
4      int i;
5      int **A = (int **)Malloc(n * sizeof(int));
6
7      for (i = 0; i < n; i++)
8          A[i] = (int *)Malloc(m * sizeof(int));
9      return A;
10 }
```

The intent here is to create an array of n pointers, each of which points to an array of m ints. However, because the programmer has written `sizeof(int)` instead of `sizeof(int *)` in line 5, the code actually creates an array of ints.

This code will run fine on machines where ints and pointers to ints are the same size. But if we run this code on a machine like the Core i7, where a pointer is larger than an int, then the loop in lines 7–8 will write past the end of the `A` array. Since one of these words will likely be the boundary-tag footer of the allocated block, we may not discover the error until we free the block much later in the program, at which point the coalescing code in the allocator will fail dramatically and for no apparent reason. This is an insidious example of the kind of “action at a distance” that is so typical of memory-related programming bugs.

9.11.5 Making Off-by-One Errors

Off-by-one errors are another common source of overwriting bugs:

```

1  /* Create an nxm array */
2  int **makeArray2(int n, int m)
3  {
4      int i;
5      int **A = (int **)Malloc(n * sizeof(int *));
6
7      for (i = 0; i <= n; i++)
8          A[i] = (int *)Malloc(m * sizeof(int));
9      return A;
10 }
```

This is another version of the program in the previous section. Here we have created an n -element array of pointers in line 5 but then tried to initialize $n + 1$ of its elements in lines 7 and 8, in the process overwriting some memory that follows the `A` array.

9.11.6 Referencing a Pointer Instead of the Object It Points To

If we are not careful about the precedence and associativity of C operators, then we incorrectly manipulate a pointer instead of the object it points to. For example, consider the following function, whose purpose is to remove the first item in a binary heap of `*size` items and then reheapify the remaining `*size - 1` items:

```

1  int *binheapDelete(int **binheap, int *size)
2  {
3      int *packet = binheap[0];
4
5      binheap[0] = binheap[*size - 1];
6      *size--; /* This should be (*size)-- */
7      heapify(binheap, *size, 0);
8      return(packet);
9  }
```

In line 6, the intent is to decrement the integer value pointed to by the `size` pointer. However, because the unary `--` and `*` operators have the same precedence and associate from right to left, the code in line 6 actually decrements the pointer itself instead of the integer value that it points to. If we are lucky, the program will crash immediately. But more likely we will be left scratching our heads when the program produces an incorrect answer much later in its execution. The moral here is to use parentheses whenever in doubt about precedence and associativity. For example, in line 6, we should have clearly stated our intent by using the expression `(*size)--`.

9.11.7 Misunderstanding Pointer Arithmetic

Another common mistake is to forget that arithmetic operations on pointers are performed in units that are the size of the objects they point to, which are not necessarily bytes. For example, the intent of the following function is to scan an array of `ints` and return a pointer to the first occurrence of `val`:

```

1  int *search(int *p, int val)
2  {
3      while (*p && *p != val)
4          p += sizeof(int); /* Should be p++ */
5      return p;
6  }
```

However, because line 4 increments the pointer by 4 (the number of bytes in an integer) each time through the loop, the function incorrectly scans every fourth integer in the array.

9.11.8 Referencing Nonexistent Variables

Naive C programmers who do not understand the stack discipline will sometimes reference local variables that are no longer valid, as in the following example:

```

1  int *stackref ()
2  {
3      int val;
4
5      return &val;
6  }
```

This function returns a pointer (say, *p*) to a local variable on the stack and then pops its stack frame. Although *p* still points to a valid memory address, it no longer points to a valid variable. When other functions are called later in the program, the memory will be reused for their stack frames. Later, if the program assigns some value to **p*, then it might actually be modifying an entry in another function's stack frame, with potentially disastrous and baffling consequences.

9.11.9 Referencing Data in Free Heap Blocks

A similar error is to reference data in heap blocks that have already been freed. Consider the following example, which allocates an integer array *x* in line 6, prematurely frees block *x* in line 10, and then later references it in line 14:

```

1  int *heapref(int n, int m)
2  {
3      int i;
4      int *x, *y;
5
6      x = (int *)Malloc(n * sizeof(int));
7
8      :   // Other calls to malloc and free go here
9      :
10     free(x);
11
12     y = (int *)Malloc(m * sizeof(int));
13     for (i = 0; i < m; i++)
14         y[i] = x[i]++; /* Oops! x[i] is a word in a free block */
15
16     return y;
17 }
```

Depending on the pattern of `malloc` and `free` calls that occur between lines 6 and 10, when the program references `x[i]` in line 14, the array *x* might be part of some other allocated heap block and may have been overwritten. As with many

memory-related bugs, the error will only become evident later in the program when we notice that the values in *y* are corrupted.

9.11.10 Introducing Memory Leaks

Memory leaks are slow, silent killers that occur when programmers inadvertently create garbage in the heap by forgetting to free allocated blocks. For example, the following function allocates a heap block *x* and then returns without freeing it:

```

1 void leak(int n)
2 {
3     int *x = (int *)Malloc(n * sizeof(int));
4
5     return; /* x is garbage at this point */
6 }
```

If *leak* is called frequently, then the heap will gradually fill up with garbage, in the worst case consuming the entire virtual address space. Memory leaks are particularly serious for programs such as daemons and servers, which by definition never terminate.

9.12 Summary

Virtual memory is an abstraction of main memory. Processors that support virtual memory reference main memory using a form of indirection known as virtual addressing. The processor generates a virtual address, which is translated into a physical address before being sent to the main memory. The translation of addresses from a virtual address space to a physical address space requires close cooperation between hardware and software. Dedicated hardware translates virtual addresses using page tables whose contents are supplied by the operating system.

Virtual memory provides three important capabilities. First, it automatically caches recently used contents of the virtual address space stored on disk in main memory. The block in a virtual memory cache is known as a page. A reference to a page on disk triggers a page fault that transfers control to a fault handler in the operating system. The fault handler copies the page from disk to the main memory cache, writing back the evicted page if necessary. Second, virtual memory simplifies memory management, which in turn simplifies linking, sharing data between processes, the allocation of memory for processes, and program loading. Finally, virtual memory simplifies memory protection by incorporating protection bits into every page table entry.

The process of address translation must be integrated with the operation of any hardware caches in the system. Most page table entries are located in the L1 cache, but the cost of accessing page table entries from L1 is usually eliminated by an on-chip cache of page table entries called a TLB.

Modern systems initialize chunks of virtual memory by associating them with chunks of files on disk, a process known as memory mapping. Memory mapping provides an efficient mechanism for sharing data, creating new processes, and loading programs. Applications can manually create and delete areas of the virtual address space using the `mmap` function. However, most programs rely on a dynamic memory allocator such as `malloc`, which manages memory in an area of the virtual address space called the heap. Dynamic memory allocators are application-level programs with a system-level feel, directly manipulating memory without much help from the type system. Allocators come in two flavors. Explicit allocators require applications to explicitly free their memory blocks. Implicit allocators (garbage collectors) free any unused and unreachable blocks automatically.

Managing and using memory is a difficult and error-prone task for C programmers. Examples of common errors include dereferencing bad pointers, reading uninitialized memory, allowing stack buffer overflows, assuming that pointers and the objects they point to are the same size, referencing a pointer instead of the object it points to, misunderstanding pointer arithmetic, referencing nonexistent variables, and introducing memory leaks.

Bibliographic Notes

Kilburn and his colleagues published the first description of virtual memory [63]. Architecture texts contain additional details about the hardware's role in virtual memory [46]. Operating systems texts contain additional information about the operating system's role [102, 106, 113]. Bovet and Cesati [11] give a detailed description of the Linux virtual memory system. Intel Corporation provides detailed documentation on 32-bit and 64-bit address translation on IA processors [52].

Knuth wrote the classic work on storage allocation in 1968 [64]. Since that time, there has been a tremendous amount of work in the area. Wilson, Johnstone, Neely, and Boles have written a beautiful survey and performance evaluation of explicit allocators [118]. The general comments in this book about the throughput and utilization of different allocator strategies are paraphrased from their survey. Jones and Lins provide a comprehensive survey of garbage collection [56]. Kernighan and Ritchie [61] show the complete code for a simple allocator based on an explicit free list with a block size and successor pointer in each free block. The code is interesting in that it uses unions to eliminate a lot of the complicated pointer arithmetic, but at the expense of a linear-time (rather than constant-time) free operation. Doug Lea developed a widely used open-source malloc package called `dlmalloc` [67].

Homework Problems

9.11 ♦

In the following series of problems, you are to show how the example memory system in Section 9.6.4 translates a virtual address into a physical address and accesses the cache. For the given virtual address, indicate the TLB entry accessed,

the physical address, and the cache byte value returned. Indicate whether the TLB misses, whether a page fault occurs, and whether a cache miss occurs. If there is a cache miss, enter “—” for “Cache byte returned.” If there is a page fault, enter “—” for “PPN” and leave parts C and D blank.

Virtual address: 0x027c

A. Virtual address format

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B. Address translation

Parameter	Value
VPN	
TLB index	
TLB tag	
TLB hit? (Y/N)	
Page fault? (Y/N)	
PPN	

C. Physical address format

[illegible]

D. Physical memory reference

Parameter	Value
Byte offset	
Cache index	
Cache tag	
Cache hit? (Y/N)	
Cache byte returned	

9.12 ♦

Repeat Problem 9.11 for the following address.

Virtual address: 0x03a9

A. Virtual address format

[illegible]

B. Address translation

Parameter	Value
VPN	
TLB index	
TLB tag	
TLB hit? (Y/N)	
Page fault? (Y/N)	
PPN	

C. Physical address format

[illegible]

D. Physical memory reference

Parameter	Value
Byte offset	00000000
Cache index	00000000
Cache tag	00000000
Cache hit? (Y/N)	00000000
Cache byte returned	00000000

9.13 ♦

Repeat Problem 9.11 for the following address.

Virtual address: 0x0040

[illegible]

A. Address translation

Parameter	Value
VPN	
TLB index	
TLB tag	
TLB hit? (Y/N)	
Page fault? (Y/N)	
PPN	

B. Physical address format

[illegible]

C. Physical memory reference

Parameter	Value
Byte offset	
Cache index	
Cache tag	
Cache hit? (Y/N)	
Cache byte returned	

9.14 ♦♦

Given an input file `hello.txt` that consists of the string `Hello, world!\n`, write a C program that uses `mmap` to change the contents of `hello.txt` to `Jello, world!\n`.

9.15 ♦

Determine the block sizes and header values that would result from the following sequence of `malloc` requests. Assumptions: (1) The allocator maintains double-word alignment and uses an implicit free list with the block format from Figure 9.35. (2) Block sizes are rounded up to the nearest multiple of 8 bytes.

Request	Block size (decimal bytes)	Block header (hex)
<code>malloc(3)</code>		
<code>malloc(11)</code>		
<code>malloc(20)</code>		
<code>malloc(21)</code>		

9.16 ♦

Determine the minimum block size for each of the following combinations of alignment requirements and block formats. Assumptions: Explicit free list, 4-byte `pred` and `succ` pointers in each free block, zero-size payloads are not allowed, and headers and footers are stored in 4-byte words.

Alignment	Allocated block	Free block	Minimum block size (bytes)
Single word	Header and footer	Header and footer	
Single word	Header, but no footer	Header and footer	
Double word	Header and footer	Header and footer	
Double word	Header, but no footer	Header and footer	

9.17 ♦♦♦

Develop a version of the allocator in Section 9.9.12 that performs a next-fit search instead of a first-fit search.

9.18 ♦♦♦

The allocator in Section 9.9.12 requires both a header and a footer for each block in order to perform constant-time coalescing. Modify the allocator so that free blocks require a header and a footer, but allocated blocks require only a header.

9.19 ♦

You are given three groups of statements relating to memory management and garbage collection below. In each group, only one statement is true. Your task is to indicate which statement is true.

1. (a) In a buddy system, up to 50% of the space can be wasted due to internal fragmentation.
 (b) The first-fit memory allocation algorithm is slower than the best-fit algorithm (on average).
 (c) Deallocation using boundary tags is fast only when the list of free blocks is ordered according to increasing memory addresses.
 (d) The buddy system suffers from internal fragmentation, but not from external fragmentation.
2. (a) Using the first-fit algorithm on a free list that is ordered according to decreasing block sizes results in low performance for allocations, but avoids external fragmentation.
 (b) For the best-fit method, the list of free blocks should be ordered according to increasing memory addresses.
 (c) The best-fit method chooses the largest free block into which the requested segment fits.
 (d) Using the first-fit algorithm on a free list that is ordered according to increasing block sizes is equivalent to using the best-fit algorithm.
3. Mark&Sweep garbage collectors are called conservative if
 - (a) They coalesce freed memory only when a memory request cannot be satisfied.
 - (b) They treat everything that looks like a pointer as a pointer.
 - (c) They perform garbage collection only when they run out of memory.
 - (d) They do not free memory blocks forming a cyclic list.

9.20 ♦♦♦♦

Write your own version of `malloc` and `free`, and compare its running time and space utilization to the version of `malloc` provided in the standard C library.

Solutions to Practice Problems

Solution to Problem 9.1 (page 805)

This problem gives you some appreciation for the sizes of different address spaces. At one point in time, a 32-bit address space seemed impossibly large. But now there are database and scientific applications that need more, and you can expect this trend to continue. At some point in your lifetime, expect to find yourself complaining about the cramped 64-bit address space on your personal computer!

Number of address bits (n)	Number of virtual addresses (N)	Largest possible virtual address
8	$2^8 = 256$	$2^8 - 1 = 255$
16	$2^{16} = 64 \text{ K}$	$2^{16} - 1 = 64 \text{ K} - 1$
32	$2^{32} = 4 \text{ G}$	$2^{32} - 1 = 4 \text{ G} - 1$
48	$2^{48} = 256 \text{ T}$	$2^{48} - 1 = 256 \text{ T} - 1$
64	$2^{64} = 16,384 \text{ P}$	$2^{64} - 1 = 16,384 \text{ P} - 1$

Solution to Problem 9.2 (page 807)

Since each virtual page is $P = 2^p$ bytes, there are a total of $2^n/2^p = 2^{n-p}$ possible pages in the system, each of which needs a page table entry (PTE).

n	$P = 2^p$	Number of PTEs
16	4 K	16
16	8 K	8
32	4 K	1 M
32	8 K	512 K

Solution to Problem 9.3 (page 816)

You need to understand this kind of problem well in order to fully grasp address translation. Here is how to solve the first subproblem: We are given $n = 32$ virtual address bits and $m = 24$ physical address bits. A page size of $P = 1 \text{ KB}$ means we need $\log_2(1 \text{ K}) = 10$ bits for both the VPO and PPO. (Recall that the VPO and PPO are identical.) The remaining address bits are the VPN and PPN, respectively.

P	Number of			
	VPN bits	VPO bits	PPN bits	PPO bits
1 KB	22	10	14	10
2 KB	21	11	13	11
4 KB	20	12	12	12
8 KB	19	13	11	13

Solution to Problem 9.4 (page 824)

Doing a few of these manual simulations is a great way to firm up your understanding of address translation. You might find it helpful to write out all the bits in the addresses and then draw boxes around the different bit fields, such as VPN, TLBI, and so on. In this particular problem, there are no misses of any kind: the TLB has a copy of the PTE and the cache has a copy of the requested data words. See Problems 9.11, 9.12, and 9.13 for some different combinations of hits and misses.

A. 00 0011 1101 0111

B. Parameter	Value
VPN	0xf
TLB index	0x3
TLB tag	0x3
TLB hit? (Y/N)	Y
Page fault? (Y/N)	N
PPN	0xd

C. 0011 0101 0111

D. Parameter	Value
Byte offset	0x3
Cache index	0x5
Cache tag	0xd
Cache hit? (Y/N)	Y
Cache byte returned	0x1d

Solution to Problem 9.5 (page 839)

Solving this problem will give you a good feel for the idea of memory mapping. Try it yourself. We haven't discussed the `open`, `fstat`, or `write` functions, so you'll need to read their man pages to see how they work.

code/vm/mmapcopy.c

```

1  #include "csapp.h"
2
3  /*
4   * mmapcopy - uses mmap to copy file fd to stdout
5   */
6  void mmapcopy(int fd, int size)
7  {
8      char *bufp; /* ptr to memory-mapped VM area */
9
10     bufp = Mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
11     Write(1, bufp, size);
12     return;
13 }
14
15 /* mmapcopy driver */
16 int main(int argc, char **argv)
17 {
18     struct stat stat;
19     int fd;
20
```

```

21      /* Check for required command-line argument */
22      if (argc != 2) {
23          printf("usage: %s <filename>\n", argv[0]);
24          exit(0);
25      }
26
27      /* Copy the input argument to stdout */
28      fd = Open(argv[1], O_RDONLY, 0);
29      fstat(fd, &stat);
30      mmapcopy(fd, stat.st_size);
31      exit(0);
32  }

```

code/vm/mmapcopy.c

Solution to Problem 9.6 (page 849)

This problem touches on some core ideas such as alignment requirements, minimum block sizes, and header encodings. The general approach for determining the block size is to round the sum of the requested payload and the header size to the nearest multiple of the alignment requirement (in this case, 8 bytes). For example, the block size for the `malloc(1)` request is $4 + 1 = 5$ rounded up to 8. The block size for the `malloc(13)` request is $13 + 4 = 17$ rounded up to 24.

Request	Block size (decimal bytes)	Block header (hex)
<code>malloc(1)</code>	8	0x9
<code>malloc(5)</code>	16	0x11
<code>malloc(12)</code>	16	0x11
<code>malloc(13)</code>	24	0x19

Solution to Problem 9.7 (page 852)

The minimum block size can have a significant effect on internal fragmentation. Thus, it is good to understand the minimum block sizes associated with different allocator designs and alignment requirements. The tricky part is to realize that the same block can be allocated or free at different points in time. Thus, the minimum block size is the maximum of the minimum allocated block size and the minimum free block size. For example, in the last subproblem, the minimum allocated block size is a 4-byte header and a 1-byte payload rounded up to 8 bytes. The minimum free block size is a 4-byte header and 4-byte footer, which is already a multiple of 8 and doesn't need to be rounded. So the minimum block size for this allocator is 8 bytes.

Alignment	Allocated block	Free block	Minimum block size (bytes)
Single word	Header and footer	Header and footer	12
Single word	Header, but no footer	Header and footer	8
Double word	Header and footer	Header and footer	16
Double word	Header, but no footer	Header and footer	8

Solution to Problem 9.8 (page 861)

There is nothing very tricky here. But the solution requires you to understand how the rest of our simple implicit-list allocator works and how to manipulate and traverse blocks.

```

1 static void *find_fit(size_t asize)
2 {
3     /* First-fit search */
4     void *bp;
5
6     for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp = NEXT_BLKP(bp)) {
7         if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp)))) {
8             return bp;
9         }
10    }
11    return NULL; /* No fit */
12 #endif
13 }

```

*code/vm/malloc/mm.c***Solution to Problem 9.9 (page 861)**

This is another warm-up exercise to help you become familiar with allocators. Notice that for this allocator the minimum block size is 16 bytes. If the remainder of the block after splitting would be greater than or equal to the minimum block size, then we go ahead and split the block (lines 6–10). The only tricky part here is to realize that you need to place the new allocated block (lines 6 and 7) before moving to the next block (line 8).

```

1 static void place(void *bp, size_t asize)
2 {
3     size_t csize = GET_SIZE(HDRP(bp));
4
5     if ((csize - asize) >= (2*DSIZE)) {
6         PUT(HDRP(bp), PACK(asize, 1));
7         PUT(FTRP(bp), PACK(asize, 1));
8         bp = NEXT_BLKP(bp);
9         PUT(HDRP(bp), PACK(csize-asize, 0));
10        PUT(FTRP(bp), PACK(csize-asize, 0));
11    }
12    else {
13        PUT(HDRP(bp), PACK(csize, 1));
14        PUT(FTRP(bp), PACK(csize, 1));
15    }
16 }

```

code/vm/malloc/mm.c